
ACTION-BED: Task-Driven Bayesian Experimental Design with Singly Intractable Objectives

Tom Rossa

Department of Statistics
University of Oxford
tom.rossa@stats.ox.ac.uk

Angus Phillips

Department of Statistics
University of Oxford
angus.phillips@stats.ox.ac.uk

Tom Rainforth

Department of Statistics
University of Oxford
rainforth@stats.ox.ac.uk

Abstract

Bayesian experimental design (BED) has traditionally been based on maximising expected uncertainty reductions from prior to posterior. A major shortfall of this approach is that it leads to doubly intractable objectives that are difficult to optimise, while customising them to particular downstream tasks of interest can also be difficult. Following first principles decision theory, we demonstrate that BED can alternatively be formulated in terms of an expected future loss (EFL) on downstream actions, providing a simple and naturally **task-driven** framework. Critically, we then show that all such EFLs can be rearranged into *singly intractable objectives* that can be jointly optimised with respect to both the design policy and a downstream *action policy* using stochastic gradients, an approach we refer to as ACTION-BED. This formulation further sidesteps the need for any explicit posterior or marginal likelihood estimation and is naturally implicit, requiring only the ability to sample from the joint model over model parameters and data, and evaluate the downstream loss function. It thus allows design policies to be learned more effectively, efficiently, and simply than existing methods, while providing easy customisation to different downstream tasks and losses.

1 Introduction

Bayesian Experimental Design (BED) [12, 17, 30, 48, 49, 58] is a principled framework for the optimal acquisition of data, in which designs are chosen to maximise the **expected uncertainty reduction** (EUR) from prior to posterior over unknown values of interest. Typically the uncertainty measure is taken to be the Shannon entropy [61], leading to the classical expected information gain (EIG) objective [48]. In this perspective, data is valued according to how it changes our belief distribution over unknown values, which is theoretically justified by the fact that downstream Bayes-optimal actions only depend on observed data through the updated belief distribution [59].

However, there are two major issues with using this framework in practice. First, defining uncertainties that are customised to particular downstream tasks of interest can be challenging [40], noting that **the data gathered is rarely the end goal in itself** and is instead used to aid downstream estimation, prediction, or decision-making. In particular, common metrics like Shannon entropy constitute generic uncertainties which fail to reflect that not all information is equally useful for our task of interest.

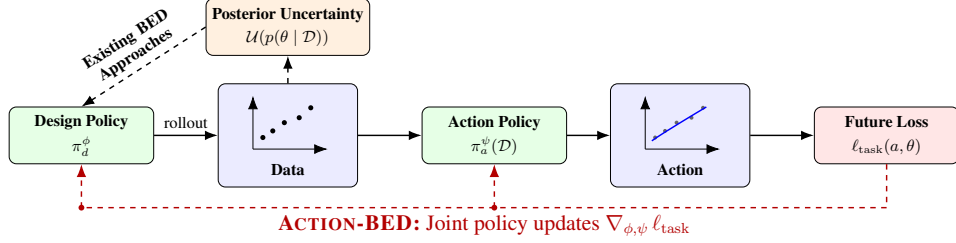


Figure 1: **ACTION-BED**; our task-driven, singly intractable algorithm for policy-based BED. Rather than targeting uncertainty reductions, ACTION-BED jointly optimises a downstream action policy alongside the design policy to minimise the expected future loss using stochastic gradient descent.

Second, EURs are generally **doubly intractable** [58]: without an analytic posterior, the EUR is inevitably a nested expectation [57] as meaningful uncertainty measures are nonlinear [16, 25, 50, 60]. This makes estimating and optimising EURs very challenging as standard estimators cannot be applied and, while a wide array of computational strategies have been proposed to deal with this [2, 21, 22, 26, 36–38], they are typically only applicable to EIGs and tend to suffer from **high computational costs and bias**. Others have instead looked to learn amortised posterior approximations to circumvent the double intractability [4, 23], often learning this alongside the design/policy optimisation using variational formulations [10, 24, 33, 34, 62]. However, this amortised posterior learning is still itself a very challenging task and inaccuracies in the posterior approximations can undermine the resulting data acquisition.

We propose to address these challenges with a fundamental move away from the EUR framework by instead defining the utility of data through *the performance of the downstream actions it enables*. Specifically, using first principles Bayesian decision theory, we show that the Bayes-optimal data gathering policy can be formulated as the minimiser of the **expected future loss** (EFL) of downstream Bayes-optimal actions, allowing for the definition of experimental design objectives that are naturally customised to our downstream tasks and losses of interest.

While the set of unique objectives indexed by the EFL and EUR frameworks turns out to be equivalent, our critical insight is that the former allows for a simple rearrangement into a *singly intractable* objective that we jointly optimise with respect to a design policy and a *downstream action policy*. This results in a much simpler and computationally tractable policy-based framework than existing approaches that we call **ACTION-BED**. As shown in Figure 1, **ACTION-BED avoids the need to directly perform posterior approximations or explicitly estimate any uncertainties: it simply requires the minimisation of an expected loss with respect to the joint policy parameters, which can be done using standard stochastic gradient descent methods**. Furthermore, the task-driven EFL objectives it uses are *naturally implicit*, requiring only reparametrisable simulations from the joint model over parameters and data, and a **differentiable** loss function. These benefits can substantially simplify the policy training and we empirically find that ACTION-BED often significantly outperforms previous state-of-the-art in terms of EIG values even when targeting different downstream tasks and losses.

In summary, we propose a paradigm shift in BED from thinking about uncertainties to future losses. Not only does this allow us to easily customise design policy learning to downstream tasks of interest, it also provides substantial computational and practical benefits, avoiding the double intractability of EURs by jointly optimising a downstream action policy alongside the design policy using our ACTION-BED framework. In turn, this provides significant empirical benefits that allow more informative and targeted data to be collected than existing state-of-the-art approaches.

2 Background

Bayesian experimental design. Bayesian experimental design (BED) [12, 17, 20, 30, 48, 49, 58] is typically defined as choosing designs to maximise the reduction in a particular uncertainty measure between the prior and the posterior. Namely, let $p(\theta)p(y|\theta, \xi)$ denote the joint generating process over outcome $y \in \mathcal{Y}$ and variables of interest $\theta \in \Theta$ given experimental design $\xi \in \Xi$. Then the expected uncertainty reduction (EUR) in θ for an uncertainty measure $\mathcal{U}$ and design ξ is:

$$\text{EUR}_\theta(\xi) = \mathbb{E}_{p(\theta)p(y|\theta, \xi)} [\mathcal{U}[p(\theta)] - \mathcal{U}[p(\theta|y, \xi)]] . \quad (1)$$

Typically, $\mathcal{U}$ is taken to be the Shannon entropy, $\mathcal{U}[q(\theta)] = \mathbb{E}_{q(\theta)} [-\log q(\theta)]$, in which case the EUR corresponds to the *expected information gain* (EIG) in θ , i.e. the mutual information between θ and y .

Sequential and policy-based BED. BED is often most impactful in sequential settings where one performs an experiment over T rounds. At each round t a design $\xi_t \in \Xi$ is proposed and outcome $y_t \in \mathcal{Y}$ is observed according to our proposed model $p(y_t|\xi_t, \theta)$. Let $h_t := \{(\xi_1, y_1), \dots, (\xi_t, y_t)\}$ be the experimental history to time t , referred to in generality as the data. Traditional approaches to sequential BED [53] would at each step infer the posterior and select the next design by optimising the EIG, however this process is prohibitively expensive. Instead, policy-based approaches [8, 10, 18, 22, 28, 33, 34, 38, 39, 62, 63] train an adaptive policy network $\pi_d \in \Pi_d$ which maps historical data h_t onto optimal next designs ξ_{t+1} . The policy is generally trained to maximise (an approximation of) the overall EUR from observing the full experimental history. This results in an acquisition policy which acts non-myopically to maximise the EUR from observing the full dataset and which can be deployed almost instantaneously. Note that the optimal design policy is always deterministic [49] and so we will assume deterministic policies throughout, but our methods can equally be applied with probabilistic design policies if desired.

Optimising EIG. The longstanding challenge with EUR objectives such as the EIG is their fundamental **double** intractability. This arises because the uncertainty measures considered are non-linear functionals of the intractable posterior distribution [16, 25, 50, 60], which are then nested within expectations over the data generating process. Much work has been devoted to developing estimation and optimisation procedures for these objectives, in particular for the EIG [2, 13, 22–24, 26, 27, 31, 32, 36–39, 42, 43, 57]. Of particular relevance is the family of variational bounds that allow the EIG to be represented as an optimisation over some kind of amortised posterior approximation [23, 24] (which is sometimes indirectly defined through a marginal likelihood approximation). For example, the Barber Agakov (BA) bound is given by [4]:

$$\mathcal{I}_T(\pi_d) \geq \mathcal{L}(\pi_d, \psi) = \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [\log q_\psi(\theta|h_T) - \log p(\theta)] \quad (2)$$

where $q_\psi(\theta|h_T)$ is an amortised approximation of the posterior distribution. **If there are some variational parameters $\psi \in \Psi$ such that $q_\psi(\theta|h_T)$ perfectly captures the true posterior $p(\theta|h_T)$ for all possible h_T , then $\max_{\psi \in \Psi} \mathcal{L}(\pi_d, \psi) = \mathcal{I}_T(\pi_d)$.** This bound then allows a singly intractable joint optimisation of both design policy π_d and variational posterior q_ψ . However, in practice, it is limited by the variational family and difficulty of learning accurate amortised posteriors.

This is the exactly why variational amortised BED is unsuitable for Neural PDE solvers. We would need to essentially relearn the "inverse" solver which sucks.

3 Loss-Driven Bayesian Experimental Design

As we have established, the de-facto perspective on BED as maximising EURs leads to objectives which are fundamentally doubly intractable and therefore pose a real computational barrier to their adoption, particularly at scale. Furthermore, variational formulations for circumventing this double intractability rely on learning accurate amortised posterior approximations, which is typically challenging and often unnecessary when the posterior itself is not the downstream goal of the experiment. They are also specific to the EIG, and there is a lack of more general computationally viable frameworks for other uncertainty measures, in particular customising to downstream tasks and losses of interest.

Instead we propose an entirely loss-driven approach, inspired by the principle that the value of data should be measured by how well it allows us to complete downstream tasks. Grounding ourselves in first principles Bayesian decision theory (BDT), we show Bayes-optimal design policies take the form of the minimisers of an expected future loss when **future actions** are made Bayes-optimally. We then show how this can be turned into a practical, computationally tractable BED algorithm, ACTION-BED, which solves the aforementioned challenges and is additionally loss-driven and **implicit**, only relying on joint simulation of parameters and data under the proposed model.

3.1 Bayesian Experimental Design from First Principles

Under **Bayesian coherence** [14], a **rational agent should plan ahead in a way that assumes that they will act optimally given the information available to them in the future**. Using this principle, we can derive a Bayes-optimal design policy from first principles, by starting with terminal actions taken after all data is gathered and then working **backwards** through our sequential decisions.

To derive the optimal downstream action given previously acquired data, we note that the Savage axioms imply the existence, for any rational agent, of a **loss function** on actions and the state of the world [59], $\ell : \mathcal{A} \times \Theta \rightarrow \mathbb{R}$, which encapsulates our preferences over the consequences from the actions we take, and a subjective **belief model** on the world given observed data $p(\theta|h_T)$.

Here $\theta \in \Theta$ now denotes whatever aspect of the true world state directly influences our loss function. The Bayes-optimal action, a^B , is then the minimiser of the Bayesian expected loss, $a^B = \arg \min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|h_T)} [\ell(a, \theta)]$. This is easily generalised to cases where the loss depends on the data h_T itself, but we omit this dependence for ease of notation.

For a given belief state and loss function, the minimal achievable Bayesian expected loss, $\min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|h_T)} [\ell(a, \theta)]$, is now a function of the data we observe. It is analogous to a value function in reinforcement learning (RL) [52]: better data will provide a more certain belief state and, in turn, a better achievable expected downstream loss. Properly formalising this requires the definition of a belief distribution on possible data that will be generated under our design policy, $p(h_T; \pi_d)$. The existence of this belief distribution is again implied for a rational agent by the Savage axioms, noting that we are uncertain about both h_T and θ before the experiment. Further, our beliefs on θ should not depend on our data gathering policy, so we should have that our “prior” $p(\theta) = \int p(h_T; \pi_d) p(\theta|h_T) dh_T$ is independent of π_d .

As minimising the loss is our ultimate objective and a rational agent will act optimally once the data is gathered, the Bayes-optimal data gathering policy is then the minimiser of the *expected future loss* (EFL) when downstream actions are themselves made Bayes-optimally as follows.

Definition 1 (Bayes-Optimal Experimental Design Policy). *The Bayes-optimal sequential experimental design policy π_d^B for a given belief model and downstream loss pairing is defined as:*

$$\pi_d^B = \arg \min_{\pi_d \in \Pi_d} \text{EFL}_\ell^B(\pi_d) \quad \text{where} \quad \text{EFL}_\ell^B(\pi_d) = \mathbb{E}_{p(h_T; \pi_d)} \left[\min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|h_T)} [\ell(a, \theta)] \right]. \quad (3)$$

We note that similar theoretical EFL formulations of the Bayes-optimal experimental designs have already been established in the classical BED literature, most notably Lindley [49] and Dawid [15]. However, they only considered individual design decisions rather than design *policies* and, moreover, it has not since been directly used for deriving computationally viable BED approaches. Instead, the BED field has since exclusively relied on the mathematical equivalence of this EFL formulation to EURs (see Section 3.3) when constructing practical methods.

3.2 Singly Intractable and Loss-Driven BED via Joint Policy Optimisation

While Definition 1 provides a precise formulation of the Bayes-optimal data gathering policy as the minimiser of a task-driven EFL, directly solving Equation (3) is very difficult as it involves a nested optimisation due to the dependence of the optimal action a^B on the observed data h_T . Our **key insight** is to switch from trying to optimise individual actions to optimising an **action policy**, π_a , which maps from observed data to downstream actions. As we show below, this leads to a singly intractable joint objective for design and action policy training which is compatible with any downstream loss.

Let $\pi_a \in \Pi_a := \{\pi \mid \pi : \mathcal{Y}^T \times \Xi^T \rightarrow \mathcal{A}\}$ be an *action policy* mapping observed data to downstream actions. Noting that Π_a contains all admissible measurable mappings from the data to actions and the optimal action is defined on a data-by-data basis for a given model and loss, under **standard measurability assumptions** detailed in Appendix A, we can lift the optimisation over individual actions in the EFL outside the expectation to become an optimisation over action policies as follows:

$$\text{EFL}_\ell^B(\pi_d) = \min_{\pi_a \in \Pi_a} \mathbb{E}_{p(h_T; \pi_d) p(\theta|h_T)} [\ell(\pi_a(h_T), \theta)]. \quad (4)$$

While the minimisation over action policies in Equation (4) does not usually have an analytic solution, we can instead think of the EFL under more general downstream action policies before jointly optimising for both π_a and π_d , which will still recover the Bayes-optimal policies for both actions:

$$(\pi_a, \pi_d)^B = \arg \min_{(\pi_a, \pi_d) \in \Pi_a \times \Pi_d} \text{EFL}_\ell(\pi_d, \pi_a), \quad (5)$$

$$\text{where } \text{EFL}_\ell(\pi_d, \pi_a) = \mathbb{E}_{p(\theta) p(y_{1:T}|\theta; \pi_d)} [\ell(\pi_a(h_T), \theta)],$$

and we have rewritten our belief model using $p(\theta) p(y_{1:T}|\theta; \pi_d) = p(h_T; \pi_d) p(\theta|h_T)$. **Prove this.**

We refer to approaches that directly train policies using Equation (5) as **ACTION-BED**. Its seemingly straightforward reinterpretation of experimental design as a joint optimisation problem now has remarkably impactful consequences. Indeed the EFL represents a **completely general, singly intractable, naturally implicit** and **loss-driven objective** for optimal experimental design. It never requires the approximation of posterior distributions or evaluation of unknown densities, it simply relies on simulations from the model. As we will see in Section 4, parameterising both action and design policies as deep neural networks leads to a practical end-to-end experimental design algorithm.

3.3 Recovering the Expected Uncertainty Reduction Formulation

Traditional EUR objectives can be formalised and reconciled with our BED formulation through the framework of **proper scoring rules**. A scoring rule is a function $s : \mathcal{P}_\theta \times \Theta \rightarrow \mathbb{R}$ which assigns a score $s(p_\theta, \theta)$ for predicting a distribution $p_\theta \in \mathcal{P}_\theta$ when the realisation of a random variable Θ is θ . The scoring rule is (strictly) **proper** if it is (strictly) minimised when the predicted distribution is exactly the distribution of the target random variable. Following ideas introduced by [7, 15, 17], we can define general uncertainty measures, known as *generalised entropies*, using $h_s[q(\theta)] = \mathbb{E}_{q(\theta)}[s(q(\cdot), \theta)]$. We then define an experimental design objective via the expected posterior uncertainty (EPU):

$$\text{EPU}_s(\pi_d) = \mathbb{E}_{p(y_{1:T}; \pi_d)} [h_s[p(\theta|h_T)]] = \mathbb{E}_{p(\theta)p(y_{1:T}|\theta; \pi_d)} [s(p_\theta(\cdot|h_T), \theta)]. \quad (6)$$

We now show that via an equivalence between loss-action pairings and proper scoring rules, any optimal BED policy according to Definition 1 can be represented as the minimiser of an EPU for some s , and a minimiser of an EPU for any s corresponds to a Bayes-optimal design policy according to Definition 1 for at least one loss-belief pairing (see Appendix B.1 for proof).

Theorem 1 (Equivalence of EFL & EPU). *Let $s : \mathcal{P}_\theta \times \Theta \rightarrow \mathbb{R}$ be a proper scoring rule on Θ . Then*

$$\pi_d^B = \arg \min_{\pi_d \in \Pi_d} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta; \pi_d)} [s(p_\theta(\cdot|h_T), \theta)] \quad (7)$$

is a Bayes-optimal design policy for the belief model $p(\theta)p(y_{1:T}|\theta; \pi_d)$ and for any loss $\ell \in \mathcal{L}$, where

$$\mathcal{L} = \left\{ \ell : \ell(\tilde{\pi}_a(q_\theta), \theta) = s(q_\theta, \theta), \forall q_\theta \in \mathcal{P}_\theta, \theta \in \Theta \right\}, \quad \tilde{\pi}_a(q_\theta) = \arg \min_{a \in \mathcal{A}} \mathbb{E}_{\theta \sim q_\theta} [\ell(a, \theta)]. \quad (8)$$

Furthermore, the set $\mathcal{L}$ is never empty, and any lower-bounded loss function ℓ induces a corresponding scoring rule s that is proper, though not necessarily strictly proper.

In order to make explicit the connection to EUR objectives, we note that if the scoring rule does not directly depend on π_d or $y_{1:T}$, the expected uncertainty of the prior $\mathbb{E}_{p(y_{1:T}; \pi_d)} [h_s[p(\theta)]] = h_s[p(\theta)]$ is constant in π_d , therefore EUR objectives are equivalent to the EPU objectives considered in Theorem 1. The set of possible EFL and EPU objectives is thus strictly greater than EURs. The EIG itself is equivalent, up to a constant offset, to an EPU objective with the log score $s(q_\theta(\cdot), \theta) = -\log q_\theta(\theta)$, **for which the optimal action turns out to be the Bayesian posterior itself.**

Theorem 1 thus reconciles the two BED paradigms as different representations of the same underlying decision problem. The key distinctions are thus first in shifting the burden of problem specification from directly choosing uncertainty measures to specifying an explicit downstream task and loss. This makes it easier to tailor objectives for specific downstream tasks, such as targeting the point estimation of some quantity of interest, or incorporating in real-world decision-making, such as business decisions or choosing medical treatments. It also makes clear how choices of uncertainty measures in the EUR framework implicitly target particular downstream tasks and losses, e.g. **EIG targets log loss on probabilistic predictions**, **expected variance reduction targets point estimation** with squared error loss.

Second, there are critical practical distinctions between the viewpoints in terms of their computational viability, with the EFL formulation leading to the general purpose singly intractable problem formulation we introduced in Equation (5). Interestingly, this formulation recovers existing variational formulations [4, 23] as a special case when using an EFL that is equivalent to the EIG (as here the optimal action is the posterior, see Appendix B.2). However, it critically generalises to other action spaces and losses, and it turns out the action policies we need to learn for other downstream losses are often much simpler than those required for the EIG. In particular, when the required downstream actions correspond to point estimates or explicit decisions rather than distributions, then the action policy is a simple functional mapping to single values, avoiding the need to implicitly estimate or approximate the posterior. For example, **if using squared error loss on an estimation task, then the action policy only needs to learn to map from data to the posterior mean**. This can substantially simplify the policy training. Indeed we observe in our experiments (Section 6) that suitably chosen predictive losses indirectly achieve higher EIG values than direct EIG optimisation approaches.

4 ACTION-BED

We now show how our ACTION-BED framework, based on the joint optimisation of the EFL with respect to action and design policies as per Equation (5), can be realised as a practical end-to-end algorithm for task-driven BED. Specifically, building on the deep adaptive design (DAD) approach of [22, 39], we propose to directly parameterise both policies using deep neural networks, denoted π_d^ϕ and π_a^ψ , with parameters $\phi \in \Phi$ and $\psi \in \Psi$. We then train these networks using **end-to-end stochastic gradient descent on the expected future loss** for rollouts drawn from the model, as shown in Algorithm 1. Empirically, it is often helpful to initialise $\psi \in \Psi$ by pre-training using a fixed random design policy (Appendix E.2), following which our algorithm remains unchanged.

Require: Policies π_d^ϕ, π_a^ψ , initialisations ϕ_0, ψ_0

- 1: **for** $k = 0, 1, \dots, K - 1$ **do**
- 2: $\theta^{1:B}, \varepsilon_{1:T}^{1:B} \sim p(\theta)q(\varepsilon_{1:T})$
- 3: $g_k^\phi, g_k^\psi \leftarrow \widehat{\nabla_{\phi, \psi}} \text{EFL}(\phi_k, \psi_k)$ (eq. (10))
- 4: $\phi_{k+1}, \psi_{k+1} \leftarrow \text{UPDATE}(\phi_k, g_k^\phi, \psi_k, g_k^\psi)$
- 5: **end for**
- 6: **return** ϕ_K, ψ_K

Algorithm 1: ACTION-BED

For example, if $p(y_{1:T}|\theta; \pi_d^\phi)$ is reparameterisable, we can optimise this objective using pathwise gradients [51] obtained via reparameterisation [41]:

$$\nabla_{\phi, \psi} \text{EFL}(\phi, \psi) = \mathbb{E}_{p(\theta)q(\varepsilon_{1:T})} \left[\nabla_{\phi, \psi} \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta) \right], \quad (9)$$

where we reparameterised $h_T = h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)$ for $\varepsilon_{1:T} \sim q(\varepsilon_{1:T})$. See Appendix C for full details of the reparameterisation. We can then estimate these gradients unbiasedly using simple Monte Carlo:

$$\widehat{\nabla_{\phi, \psi}} \text{EFL}(\phi, \psi) = \frac{1}{B} \sum_{b=1}^B \nabla_{\phi, \psi} \ell(\pi_a^\psi(h_T(\theta^b, \varepsilon_{1:T}^b; \pi_d^\phi)), \theta^b) \quad (10)$$

for $\theta^b \sim p(\theta), \varepsilon_{1:T}^b \sim q(\varepsilon_{1:T})$, from which stochastic gradient updates can be performed using any desired optimizer. If the simulator is not reparameterisable, we can resort to the REINFORCE estimator [66] to estimate gradients, provided that we have access to the score function of the data generating process. These requirements are the main limitations of our approach, although we note that a very large class of BED problems satisfy at least one of these assumptions. Details on the architectures used for the policies are provided in Appendix E.

5 Related Work

To the best of our knowledge, there are very limited prior works considering **truly task-driven approaches** to BED. One approach is Huang et al. [33], who maximise the improvement in expected utility of the optimal downstream action after observing new data. They only consider utility functions on actions living in outcome space $\mathcal{Y}$, meaning their method cannot be applied as a baseline in any of our BED experiments and is instead applied on **prediction-oriented** active learning settings [65]. Furthermore, they maximise their expected utility objective by approximating the intractable predictive distribution and explicitly solving the nested optimisation problem over downstream actions. In contrast, our approach does not require any distributional approximations and removes the nested optimisation by learning action policies. Short of being truly task-based, recent works [11, 19, 44, 62, 68, 69] consider goal-oriented experimental design, in which designs are sought to maximise the EUR in downstream quantities of interest. While this approach shifts focus from model parameters, it is still fundamentally an EUR objective and therefore inherits the associated computation challenges. Finally, Kerrigan et al. [40] introduces a new perspective on BED, maximising the dependence between y and θ by solving an optimal transport problem with a chosen cost function. The choice of cost function provides a way of tailoring designs to downstream tasks of interest, but cannot be interpreted as minimising a loss over downstream actions in a principled BDT perspective.

In alternative contexts, Huang et al. [35] introduces a task-driven framework for active learning, which they solve analytically when the downstream loss is a Bregman divergence. Our approach is instead singly intractable for *any* downstream loss. In a Bayesian optimisation (BO) setting, Neiswanger et al. [55] frame classical BO acquisition functions in a loss-based perspective via generalised entropies, which they use to derive new acquisition functions for custom BO tasks. In their case, analytic solutions depend on the Gaussian process posterior, specific to the BO setting.

Our approach also shares strong connections with RL, since $\min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|h_T)} [\ell(a, \theta)]$ can be interpreted as a value function giving the best achievable downstream loss given the current belief state. Prior works [5, 8, 9, 62, 63] have adopted an RL approach to BED by defining rewards as the incremental improvement in the above value function. However, this has only been done for the EIG, implicitly choosing the log-loss to define the value function, while the generalisation to any downstream loss has not yet been made. Furthermore, while variational and amortised inference approaches [10, 18, 24, 34, 62] learn amortised variational approximations which can be interpreted as downstream action networks (often within RL frameworks e.g. Shen et al. [62]), the generalisation to arbitrary downstream actions has also not yet been made. Ultimately our approach could directly benefit from inheriting RL ideas, such as reward shaping and credit assignment, in the future.

6 Experiments

We evaluate **ACTION-BED** across four BED tasks spanning a range of complementary settings. Our experiments illustrate the improved performance and computational efficiency of ACTION-BED on downstream tasks, due to our task-driven and singly intractable procedure.

Baselines and evaluation protocol. We compare against two competitive policy-based BED baselines. **DAD** [22] serves as the standard EIG-based policy baseline when an explicit likelihood is available, while **IDAD** [39] provides its likelihood-free counterpart. **ALINE** [34] is a gold-standard EIG-based approach using a transformer as a joint amortised design and inference network, trained using reinforcement learning with a dense reward. Additionally we include a **Random** policy which samples designs from a task-specific proposal. In order to evaluate these baselines on downstream actions, we train a design policy using the baseline approach, which is then frozen while a downstream action network is trained to minimise the downstream task loss on trajectories generated by the pre-trained design policy. Thus, the design network of each baseline is trained to maximise EIG while the downstream network is trained to minimise downstream loss by approximating the Bayes-optimal downstream action. Methods are evaluated in terms of downstream loss, design policy EIG bounds [22], and end-to-end wall-clock GPU training time.

Training robustness. To confirm that our relative performance gains are not driven by architectural choices, we matched architectures across all baselines and further repeated our experiments using smaller architectures in Appendix G.4. Due to limited computational resources, our main results for the source location finding benchmark are presented for a single training seed. We therefore evaluate training stability using the smaller architectures, with results reported in Appendix G.4.2 and Table 33, showing our downstream performance is stable over training seeds.

6.1 Source Location Finding

The first experiment is inspired by the acoustic energy attenuation model introduced in [64]. The unknown parameter $\theta \in \mathbb{R}^{2 \times 2}$ specifies the locations of two sources which emit signals that decay with distance. Over $T = 30$ steps, the design policy sequentially selects locations and observes the resulting aggregate signal intensity. The goal is to learn an adaptive design strategy that enables efficient downstream prediction of source locations. For the downstream predictive loss, we consider the mean squared error (MSE) and the Log-MSE, shorthand for an MSE loss augmented with a logarithmic penalty, $\ell_{\text{Log-MSE}}(\theta, \hat{\theta}) = \|\theta - \hat{\theta}\|^2 + \log(\|\theta - \hat{\theta}\|^2 + \epsilon)$, which increases incentive for precise localisation. Implementation, architectural, and experimental details are provided in Appendix G.1.

Results. Table 1 shows the quantitative results on the source location finding task while qualitative design trajectories are included in Appendix G.1.1. We firstly observe the remarkable performance of ACTION-BED under the Log-MSE objective; we achieve both the best downstream Log-MSE error as well as state-of-the-art EIG of the design policy. We believe that this is possible because the Log-MSE remains sensitive to very accurate predictions and continues to reward design policies that identify the sources more precisely, whereas the sPCE contrastive lower bound saturates at

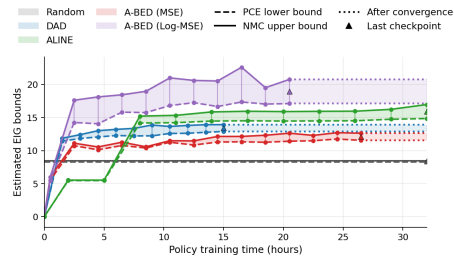


Figure 2: Comparison of EIG bounds as a function of design policy training time, until convergence.

Table 1: **Source Location Finding.** Uncertainties are reported as one standard error over 2048 rollouts. Parentheses indicate the loss used to train the downstream action network. Compute time includes training both the design and downstream action networks.

Method	Performance (± 1 s.e.)				Compute
	MSE (10^{-2}) $\downarrow$	Log-MSE $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$	Time (min) $\downarrow$
Action-BED (MSE)	0.55 ± 0.06	-6.83 ± 0.03	11.63 ± 0.04	12.84 ± 0.08	1171
Action-BED (Log)	2.8 ± 0.4	-10.10 ± 0.03	17.14 ± 0.11	21.07 ± 0.45	1184
DAD (MSE)	5.6 ± 0.6	-6.35 ± 0.04	12.94 ± 0.03	16.35 ± 0.14	1214
DAD (Log)	10.7 ± 0.8	-6.58 ± 0.05	12.94 ± 0.03	16.35 ± 0.14	1216
ALINE (MSE)	0.21 ± 0.03	-7.58 ± 0.05	14.83 ± 0.05	16.97 ± 0.13	2962
ALINE (Log)	0.81 ± 0.10	-8.11 ± 0.07	14.83 ± 0.05	16.97 ± 0.13	2970
Random	22.4 ± 0.8	-3.16 ± 0.04	8.27 ± 0.04	8.49 ± 0.05	31

high EIG values therefore becoming harder to train to. Utilising this informative loss function and sidestepping the challenges of EIG-based training is made directly possible by our approach. Under the MSE loss, ALINE performs best however requires around **2.5 times the compute** to achieve such a result. A plausible explanation is that its dense training rewards help identify the latent sources more efficiently, while such guidance is absent in our setting. ACTION-BED significantly outperforms DAD, which uses a similar amount of compute. In terms of computational efficiency, we show in Figure 2 the attained EIG of the design policy versus training time, where ACTION-BED (Log) achieves competitive EIG values faster than competing approaches.

Additional ablations and evaluation. We provide several additional analyses in Appendix G.1.2 and G.1.3. We first evaluate the decision-relevant quality of the learned design policies independently of their associated downstream action networks by applying a common posterior-based decision procedure to all methods. This isolates the informativeness of the acquired histories themselves and confirms the objective-dependent ranking observed in the main results. Moreover, it highlights the substantial deployment-time cost avoided by amortised decision-making with ACTION-BED; see Tables 16 and 17. We then investigate the coupling between the jointly learned design policy and downstream action policy in ACTION-BED by evaluating each design and action network cross-pairing; see Table 20. Downstream performance degrades substantially under such policy swaps, showing that the action network exploits not only the observed outcomes but also information encoded in the adaptive design trajectory itself. This coupling is strongest for the most accurate policies, whose designs concentrate efficiently near the latent sources. Second, we show for ACTION-BED that retraining the action policy on a fixed pre-trained design policy leads to a mild downstream degradation compared to joint training, suggesting that the joint dynamics beneficially co-adapt the two policies; see Table 19. Finally, we ablate the warmstart used before joint optimisation; see Table 21. Removing it slightly worsens downstream performance, likely because early low-quality action predictions provide a noisier training signal to the design policy, perturbing the joint optimisation dynamics. In this benchmark, warmstarting is therefore useful and incurs only a small computational overhead.

6.2 Dynamical Systems

We next consider stochastic **single- and double-pendulum systems** [6, 38]. In these tasks, the system evolves over time according to a controlled SDE depending on some unknown physical parameters. The continuous dynamics are discretised using Euler-Maruyama over $T = 50$ steps with $dt = 0.05$. At each step an agent influences the dynamics by applying a torque ξ_t . The goal is to learn an adaptive design policy that enables accurate downstream prediction of the latent physical parameters $\theta \in \Theta$ from the observed trajectories. We evaluate the prediction with several downstream objectives—MSE, log-MSE, and a weighted MSE over the components of θ —the latter requiring the design policy to prioritise parameter regions of particular interest. These tasks pose three main challenges: the data-generating process is **non-exchangeable** due to Markovian dynamics, designs must satisfy physical constraints, and longer rollouts induce complex feedback between interventions and observations.

Stochastic Pendulum. This task comprises a single pendulum with state $y_t = (q_t, \dot{q}_t)$ and unknown parameters $\theta \in \Theta \subseteq \mathbb{R}^3$. At each step, the agent applies a bounded torque $\xi_t \in [-1, 1]$ to control the system. Full experimental details are provided in Appendix G.2.1. Table 2 confirms that loss-driven acquisition improves the corresponding downstream objective: each variant performs best under the

Table 2: **Stochastic Single Pendulum.** Uncertainties are reported as one standard error over 2048 rollouts. Compute time includes training both the design and downstream action networks.

Method	Performance (± 1 s.e.)					Compute
	MSE (10^{-3}) $\downarrow$	Log-MSE $\downarrow$	W. MSE (10^{-3}) $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$	Time (min) $\downarrow$
Action-BED (MSE)	6.00 $\pm$ 0.14	-6.60 ± 0.05	4.90 ± 0.13	3.68 ± 0.04	3.71 ± 0.04	240
Action-BED (Log)	7.20 ± 0.17	-6.75 ± 0.05	4.80 ± 0.11	3.99 $\pm$ 0.04	4.00 $\pm$ 0.04	287
Action-BED (W. MSE)	9.80 ± 0.25	-6.07 ± 0.04	3.50 $\pm$ 0.09	3.33 ± 0.04	3.36 ± 0.04	290
DAD (MSE)	7.20 ± 0.17	-6.20 ± 0.04	5.00 ± 0.14	3.82 ± 0.04	3.88 ± 0.04	670
DAD (Log)	8.10 ± 0.19	-6.49 ± 0.05	5.20 ± 0.14	3.82 ± 0.04	3.88 ± 0.04	709
DAD (W. MSE)	7.30 ± 0.17	-6.44 ± 0.05	4.40 ± 0.12	3.82 ± 0.04	3.88 ± 0.04	671
ALINE (MSE)	23.50 ± 0.51	-5.43 ± 0.02	7.59 ± 0.12	2.23 ± 0.06	2.24 ± 0.06	2120
ALINE (Log)	27.53 ± 0.59	-5.69 ± 0.02	9.89 ± 0.17	2.23 ± 0.06	2.24 ± 0.06	2141
ALINE (W. MSE)	23.58 ± 0.50	-5.61 ± 0.02	7.06 ± 0.11	2.23 ± 0.06	2.24 ± 0.06	2122
Random	29.0 ± 1.2	-4.25 ± 0.05	12.6 ± 1.0	1.42 ± 0.03	1.50 ± 0.03	19

Table 3: **Stochastic Double Pendulum.** Uncertainties are reported as one standard error over 2048 rollouts. Compute time includes training both the design and downstream action networks.

Method	Performance (± 1 s.e.)					Compute
	MSE (10^{-4}) $\downarrow$	Log-MSE $\downarrow$	W. MSE (10^{-4}) $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$	Time (min) $\downarrow$
Action-BED (MSE)	1.11 $\pm$ 0.03	-10.21 ± 0.04	1.64 ± 0.04	10.73 ± 0.05	12.71 ± 0.19	501
Action-BED (Log)	1.31 ± 0.03	-10.32 ± 0.04	1.75 ± 0.05	10.81 ± 0.12	14.29 ± 0.56	504
Action-BED (W. MSE)	1.14 ± 0.03	-10.28 ± 0.04	1.51 $\pm$ 0.05	10.65 ± 0.04	12.10 ± 0.16	504
DAD (MSE)	2.10 ± 0.06	-9.50 ± 0.03	2.47 ± 0.07	11.37 $\pm$ 0.03	14.79 $\pm$ 0.25	934
DAD (Log)	2.43 ± 0.10	-9.51 ± 0.03	2.79 ± 0.08	11.37 $\pm$ 0.03	14.79 $\pm$ 0.25	940
DAD (W. MSE)	2.13 ± 0.06	-9.47 ± 0.03	2.46 ± 0.07	11.37 $\pm$ 0.03	14.79 $\pm$ 0.25	933
ALINE (MSE)	11.21 ± 0.34	-8.46 ± 0.02	15.12 ± 0.04	9.28 ± 0.07	9.78 ± 0.14	3013
ALINE (Log)	19.21 ± 0.51	-8.53 ± 0.03	23.29 ± 0.07	9.28 ± 0.07	9.78 ± 0.14	3062
ALINE (W. MSE)	11.81 ± 0.33	-8.52 ± 0.03	14.02 ± 0.04	9.28 ± 0.07	9.78 ± 0.14	3007
Random	20.0 ± 0.9	-6.84 ± 0.05	48.0 ± 1.3	8.26 ± 0.06	8.41 ± 0.08	58

loss used for training. Interestingly, we observe again that the design policy of ACTION-BED (Log) achieves the highest EIG, outperforming the baselines which directly optimise EIG. We also observe that our method requires substantially less compute than the baselines.

Stochastic Double-Pendulum. The double-pendulum task is substantially harder as its dynamics are nonlinear, coupled, and potentially chaotic, making trajectories highly sensitive to designs. The state $y_t \in \mathbb{R}^4$ contains the two joint angles and angular velocities, while $\theta \in \Theta \subseteq \mathbb{R}^4$ contains the **unknown masses and link lengths**. Designs are bounded joint torques $\xi_t = (\xi_t^1, \xi_t^2) \in [-4, 4] \times [-2, 2]$. Details are given in Appendix G.2.2. In this setting, Table 3 again shows that ACTION-BED achieves considerably improved downstream prediction for reduced computational cost. In this task, DAD appears to achieve marginally higher EIG although EIG bounds overlap with ACTION-BED.

6.3 MNIST Masked Classification

Finally, we consider a sequential masked-classification task adapted from the experimental setting in [37]. Each episode begins with an unobserved image $\theta \in \mathbb{R}^{28 \times 28}$ from MNIST [46], together with its class label $z \in \{0, \dots, 9\}$. At each of the $T = 5$ acquisition steps, the policy selects the corner $\xi \in [1, 28]^2$ of a local 5×5 patch and observes a noisy measurement of that patch, $y = A_\xi \theta + \eta$, with $\eta \sim \mathcal{N}(0, \sigma I_d)$. Here, $A_\xi \theta$ denotes the masking operator induced by ξ , and applied to the image. The goal is to select the patches most informative for downstream digit classification. This setting is challenging because **acquisition is driven by the class label**, whereas measurements arise from the latent image. The patch likelihood $p(y|\theta, \xi)$ is **explicit**, but its label-conditioned counterpart $p(y|z, \xi)$ is only **implicitly** induced by the MNIST class distribution. This problem is also strongly information-constrained, since the downstream policy must infer the class from only a few noisy local measurements in a high-dimensional space. Additional details are provided in Appendix G.3.

Table 4: **MNIST Masked Classification.** Uncertainties are reported as one standard error across evaluation batches. We report cross-entropy and classification accuracy on both the training and test datasets. GPU time includes training of both the design and downstream classification networks.

Method	Train dataset		Test dataset		Compute
	Cross-entropy	Accuracy (%)	Cross-entropy	Accuracy (%)	GPU time (min)
Action-BED	0.033 ± 0.001	98.87 ± 0.04	0.097 ± 0.007	97.42 ± 0.16	9
DAD	0.350 ± 0.004	83.32 ± 0.04	0.546 ± 0.009	81.83 ± 0.16	53
iDAD	0.099 ± 0.002	96.49 ± 0.08	0.164 ± 0.008	95.10 ± 0.21	12
ALINE	0.256 ± 0.008	92.13 ± 0.08	0.271 ± 0.01	91.59 ± 0.10	357
Random	0.786 ± 0.015	71.78 ± 0.87	0.797 ± 0.015	71.40 ± 0.86	5

ACTION-BED naturally targets this implicit problem by jointly training a design policy and downstream classifier to minimise the cross-entropy loss:

$$\min_{\phi, \psi} \mathbb{E}_{p(z)p(\theta|z)p(y_{1:T}|\theta;\pi_d^\phi)} [\ell_{\text{CE}}(\pi_a^\psi(\cdot|h_T), z)]$$

where $\ell_{\text{CE}}(\pi_a^\psi(\cdot|h_T), z) = -\log \pi_a^\psi(z|h_T)$ is the cross-entropy loss and $\pi_a^\psi(\cdot|h_T)$ is a probabilistic classifier. Training to this objective only requires simulating z, θ from the empirical MNIST dataset and **reparameterising** the simulation of $p(y_{1:T}|\theta;\pi_d)$. By contrast, DAD must target EIG in the latent image θ , since the outcomes conditional on class label $p(y|z, \xi)$ do not have explicit likelihood. iDAD is able to target the EIG in the label z , thereby providing a more competitive baseline. ALINE does not naturally apply in this setting due to the hierarchical likelihood and classification target, we therefore adapted the setup of the inference network accordingly. Quantitative and qualitative results are reported respectively in Table 4 and Figure 7. ACTION-BED achieves the best downstream performance across the board, demonstrating the lowest cross-entropy loss and highest classification accuracy amongst all approaches, whilst requiring the least training time. These results demonstrate that ACTION-BED easily maximises downstream performance in implicit models without additional modifications, critic networks, or **internal posterior approximations**.

7 Conclusion

In this work we introduce ACTION-BED, a practical algorithm for task-driven BED, resulting from the simple principle of minimising an expected future loss over downstream actions. We show that this approach is grounded in first principles decision theory and we demonstrate an equivalence to existing EUR approaches. Our algorithm, which jointly learns a design network and downstream action network, is singly intractable, naturally implicit and directly targets downstream task performance. As a result, we demonstrate improved downstream performance compared to baselines which directly optimise for EIG, while requiring significantly less training effort. Our approach is applicable to BED problems with reparameterisable simulators or access to gradients of the log-likelihood of the observation model, which represents a very broad class of BED problems. Advancing the efficiency of BED methods for downstream task performance has clear societal benefits by enabling better decision making and downstream predictions whenever these actions are informed by experimental data, for instance in medical diagnoses. While experimental data can sometimes be collected with malicious intent, we believe the potential benefits of our approach outweigh the risks of misuse.

Acknowledgements

Tom Rossa is supported by the EPSRC CDT in Statistics and Machine Learning (EP/Y034813/1). Angus Phillips is supported by the EPSRC CDT in Modern Statistics and Statistical Machine Learning (EP/S023151/1). Tom Rainforth is supported by the EPSRC grant EP/Y037200/1.

References

- [1] Aliprantis, C. D. and Border, K. C. (2006). *Infinite Dimensional Analysis: A Hitchhiker's Guide*. Springer, 3 edition. 16
- [2] Ao, Z. and Li, J. (2024). On Estimating the Gradient of the Expected Information Gain in Bayesian Experimental Design. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(18):20311–20319. Number: 18. 2, 3
- [3] Ashman, M., Diaconu, C., Weller, A., and Turner, R. E. (2024). In-context in-context learning with transformer neural processes. In Antorán, J. and Naesseth, C. A., editors, *Proceedings of the 6th Symposium on Advances in Approximate Bayesian Inference*, volume 253 of *Proceedings of Machine Learning Research*, pages 1–29. PMLR. 23
- [4] Barber, D. and Agakov, F. (2003). Information Maximization in Noisy Channels : A Variational Approach. In *Advances in Neural Information Processing Systems*, volume 16. MIT Press. 2, 3, 5
- [5] Barlas, Y. and Salako, K. (2025). Performance Comparisons of Reinforcement Learning Algorithms for Sequential Experimental Design. Philadelphia, USA. 7
- [6] Belousov, B., Abdulsamad, H., Schultheis, M., and Peters, J. (2019). Belief space model predictive control for approximately optimal system identification. In *Proceedings of the Multidisciplinary Conference on Reinforcement Learning and Decision Making (RLDM)*. 8
- [7] Bickford Smith, F., Kossen, J., Trollope, E., Van Der Wilk, M., Foster, A., and Rainforth, T. (2025). Rethinking aleatoric and epistemic uncertainty. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 4345–4359. PMLR. 5
- [8] Blau, T., Bonilla, E. V., Chades, I., and Dezfouli, A. (2022). Optimizing Sequential Experimental Design with Deep Reinforcement Learning. In *Proceedings of the 39th International Conference on Machine Learning*, pages 2107–2128. PMLR. 3, 7
- [9] Blau, T., Chades, I., Dezfouli, A., Steinberg, D. M., and Bonilla, E. V. (2023). Cross-Entropy Estimators for Sequential Experiment Design with Reinforcement Learning. 7
- [10] Bracher, N., Kühmichel, L., Ivanova, D. R., Intes, X., Bürkner, P.-C., and Radev, S. T. (2025). JADAI: Jointly Amortizing Adaptive Design and Bayesian Inference. arXiv:2512.22999 [stat]. 2, 3, 7
- [11] Chakraborty, A., Huan, X., and Catanach, T. A. (2024). A Likelihood-Free Approach to Goal-Oriented Bayesian Optimal Experimental Design. 6
- [12] Chaloner, K. and Verdinelli, I. (1995). Bayesian Experimental Design: A Review. *Statistical Science*, 10(3):273–304. 1, 2
- [13] Coons, T. E. and Huan, X. (2025). A Multi-fidelity Estimator of the Expected Information Gain for Bayesian Optimal Experimental Design. arXiv:2501.10845 [stat]. 3
- [14] Cox, R. T. et al. (1946). Probability, frequency and reasonable expectation. *American journal of physics*, 14(1):1–13. 3
- [15] Dawid, A. P. (1998). Coherent measures of discrepancy, uncertainty and dependence, with applications to bayesian predictive experimental design. *Department of Statistical Science, University College London*. <http://www.ucl.ac.uk/Stats/research/abs94.html>, Tech. Rep, 139. 4, 5
- [16] Dawid, A. P. (2007). The geometry of proper scoring rules. *Annals of the Institute of Statistical Mathematics*, 59(1):77–93. 2, 3
- [17] DeGroot, M. (1962). Uncertainty, information, and sequential experiments. *The Annals of Mathematical Statistics*, 33(2):404–419. 1, 2, 5
- [18] Dong, J., Jacobsen, C., Khalloufi, M., Akram, M., Liu, W., Duraisamy, K., and Huan, X. (2025). Variational Bayesian optimal experimental design with normalizing flows. *Computer Methods in Applied Mechanics and Engineering*, 433:117457. 3, 7

- [19] Filstroff, L., Sundin, I., Mikkola, P., Tiulpin, A., Kylmäoja, J., and Kaski, S. (2024). Targeted Active Learning for Bayesian Decision-Making. *Transactions on Machine Learning Research*. 6
- [20] Fisher, R. (1935). The design of experiments. 2
- [21] Fort, G., Gobet, E., and Moulines, E. (2017). Mcmc design-based non-parametric regression for rare event. application to nested risk computations. *Monte Carlo Methods and Applications*, 23(1):21–42. 2
- [22] Foster, A., Ivanova, D. R., Malik, I., and Rainforth, T. (2021). Deep Adaptive Design: Amortizing Sequential Bayesian Experimental Design. In *Proceedings of the 38th International Conference on Machine Learning*, pages 3384–3395. PMLR. 2, 3, 6, 7, 23, 29, 30, 48
- [23] Foster, A., Jankowiak, M., Bingham, E., Horsfall, P., Teh, Y. W., Rainforth, T., and Goodman, N. (2019a). Variational Bayesian Optimal Experimental Design. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc. 2, 3, 5, 18
- [24] Foster, A., Jankowiak, M., O’Meara, M., Teh, Y., and Rainforth, T. (2019b). A Unified Stochastic Gradient Approach to Designing Bayesian-Optimal Experiments. 2, 3, 7, 18
- [25] Gneiting, T. and Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American statistical Association*, 102(477):359–378. 2, 3
- [26] Goda, T., Hironaka, T., and Iwamoto, T. (2020). Multilevel Monte Carlo estimation of expected information gains. *Stochastic Analysis and Applications*, 38(4):581–600. _eprint: <https://doi.org/10.1080/07362994.2019.1705168>. 2, 3
- [27] Goda, T., Hironaka, T., Kitade, W., and Foster, A. (2022). Unbiased MLMC Stochastic Gradient-Based Optimization of Bayesian Experimental Designs. *SIAM Journal on Scientific Computing*, 44(1):A286–A311. 3
- [28] Hedman, M., Ivanova, D. R., Guan, C., and Rainforth, T. (2025). Step-DAD: Semi-Amortized Policy-Based Bayesian Experimental Design. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 22904–22923. PMLR. 3
- [29] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780. 23
- [30] Huan, X., Jagalur, J., and Marzouk, Y. (2024). Optimal experimental design: Formulations and computations. *Acta Numerica*, 33:715–840. 1, 2
- [31] Huan, X. and Marzouk, Y. (2012). Gradient-based stochastic optimization methods in Bayesian experimental design. *International Journal for Uncertainty Quantification*, 4. 3
- [32] Huan, X. and Marzouk, Y. M. (2013). Simulation-based optimal Bayesian experimental design for nonlinear systems. *J. Comput. Phys.*, 232(1):288–317. 3
- [33] Huang, D., Guo, Y., Acerbi, L., and Kaski, S. (2024). Amortized Bayesian Experimental Design for Decision-Making. 2, 3, 6
- [34] Huang, D., Wen, X., Bharti, A., Kaski, S., and Acerbi, L. (2025). ALINE: Joint Amortization for Bayesian Inference and Active Data Acquisition. 2, 3, 7
- [35] Huang, Z., Smith, F. B., and Rainforth, T. (2026). Loss-Driven Bayesian Active Learning. arXiv:2604.11995 [cs]. 6
- [36] Iollo, J., Heinkelé, C., Alliez, P., and Forbes, F. (2024). PASOA- PArticle baSed Bayesian Optimal Adaptive design. In *Proceedings of the 41st International Conference on Machine Learning*, pages 21020–21046. PMLR. 2, 3
- [37] Iollo, J., Heinkelé, C., Alliez, P., and Forbes, F. (2025). Bayesian Experimental Design via Contrastive Diffusions. arXiv:2410.11826 [stat] version: 2. 9, 44

- [38] Iqbal, S., Corenflos, A., Särkkä, S., and Abdulsamad, H. (2024). Nesting Particle Filters for Experimental Design in Dynamical Systems. In *Proceedings of the 41st International Conference on Machine Learning*, pages 21047–21068. PMLR. 2, 3, 8, 39
- [39] Ivanova, D. R., Foster, A., Kleinegesse, S., Gutmann, M. U., and Rainforth, T. (2021). Implicit Deep Adaptive Design: Policy-Based Experimental Design without Likelihoods. In *Advances in Neural Information Processing Systems*, volume 34, pages 25785–25798. Curran Associates, Inc. 3, 6, 7, 23, 50
- [40] Kerrigan, G., Naesseth, C. A., and Rainforth, T. (2025). A Geometric Approach to Optimal Experimental Design. arXiv:2510.14848 [stat]. 1, 6
- [41] Kingma, D. P. and Welling, M. (2013). Auto-Encoding Variational Bayes. arXiv:1312.6114 [stat]. 6
- [42] Kleinegesse, S. and Gutmann, M. U. (2019). Efficient Bayesian Experimental Design for Implicit Models. In *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, pages 476–485. PMLR. 3
- [43] Kleinegesse, S. and Gutmann, M. U. (2020). Bayesian Experimental Design for Implicit Models by Mutual Information Neural Estimation. In *Proceedings of the 37th International Conference on Machine Learning*, pages 5316–5326. PMLR. 3
- [44] Kleinegesse, S. and Gutmann, M. U. (2021). Gradient-based Bayesian Experimental Design for Implicit Models using Mutual Information Lower Bounds. arXiv:2105.04379 [stat]. 6
- [45] Kuratowski, K. and Ryll-Nardzewski, C. (1965). A general theorem on selectors. *Bull. Acad. Polon. Sci. Sér. Sci. Math. Astronom. Phys*, 13(6):397–403. 16
- [46] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324. 9, 44
- [47] Lee, J., Lee, Y., Kim, J., Kosiorek, A. R., Choi, S., and Teh, Y. W. (2019). Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*. 23
- [48] Lindley, D. V. (1956). On a Measure of the Information Provided by an Experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005. 1, 2
- [49] Lindley, D. V. (1972). *Bayesian statistics*. Society for Industrial and Applied Mathematics. tex.eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611970654>. 1, 2, 3, 4
- [50] Lindley, D. V. (1982). Scoring rules and the inevitability of probability. *International statistical review/revue internationale de statistique*, pages 1–11. 2, 3
- [51] Mohamed, S., Rosca, M., Figurnov, M., and Mnih, A. (2020). Monte Carlo Gradient Estimation in Machine Learning. *Journal of Machine Learning Research*, 21(132):1–62. 6
- [52] Murphy, K. (2024). Reinforcement learning: an overview. *arXiv preprint arXiv:2412.05265*. 4
- [53] Myung, J. I., Cavagnaro, D. R., and Pitt, M. A. (2013). A tutorial on adaptive design optimization. *Journal of Mathematical Psychology*, 57(3):53–67. 3
- [54] Neal, R. M. (2011). Mcmc using hamiltonian dynamics. In Brooks, S., Gelman, A., Jones, G. L., and Meng, X.-L., editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC. 34
- [55] Neiswanger, W., Yu, L., Zhao, S., Meng, C., and Ermon, S. (2022). Generalizing Bayesian optimization with decision-theoretic entropies. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, pages 21016–21029, Red Hook, NY, USA. Curran Associates Inc. 6

- [56] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché Buc, F., Fox, E., and Garnett, R., editors, *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035. 29
- [57] Rainforth, T., Cornish, R., Yang, H., Warrington, A., and Wood, F. (2018). On Nesting Monte Carlo Estimators. In *Proceedings of the 35th International Conference on Machine Learning*, pages 4267–4276. PMLR. 2, 3
- [58] Rainforth, T., Foster, A., Ivanova, D. R., and Smith, F. B. (2024). Modern Bayesian Experimental Design. *Statistical Science*, 39(1):100–114. 1, 2
- [59] Savage, L. J. (1951). *The theory of statistical decision*, volume 46. Journal of the American Statistical Association. 1, 3
- [60] Savage, L. J. (1971). Elicitation of personal probabilities and expectations. *Journal of the American Statistical Association*, 66(336):783–801. 2, 3
- [61] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3):379–423. _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/j.1538-7305.1948.tb01338.x>. 1
- [62] Shen, W., Dong, J., and Huan, X. (2025). Variational sequential optimal experimental design using reinforcement learning. *Computer Methods in Applied Mechanics and Engineering*, 444:118068. 2, 3, 6, 7
- [63] Shen, W. and Huan, X. (2023). Bayesian Sequential Optimal Experimental Design for Nonlinear Models Using Policy Gradient Reinforcement Learning. *Computer Methods in Applied Mechanics and Engineering*, 416:116304. arXiv:2110.15335 [cs]. 3, 7
- [64] Sheng, X. and Hu, Y.-H. (2005). Maximum likelihood multiple-source localization using acoustic energy measurements with wireless sensor networks. *IEEE Transactions on Signal Processing*, 53(1):44–53. 7
- [65] Smith, F. B., Kirsch, A., Farquhar, S., Gal, Y., Foster, A., and Rainforth, T. (2023). Prediction-Oriented Bayesian Active Learning. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, pages 7331–7348. PMLR. 6
- [66] Williams, R. J. (1992). Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Mach. Learn.*, 8(3-4):229–256. 6
- [67] Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., and Smola, A. J. (2017). Deep sets. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R., editors, *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 3391–3401. Curran Associates, Inc. 23
- [68] Zhang, Z., Dong, J., Liu, J., and Huan, X. (2025). Goal-oriented sequential bayesian experimental design for causal learning. 6
- [69] Zhong, S., Shen, W., Catanach, T., and Huan, X. (2026). Goal-Oriented Bayesian Optimal Experimental Design for Nonlinear Models using Markov Chain Monte Carlo. *SIAM/ASA Journal on Uncertainty Quantification*, 14(1):19–47. arXiv:2403.18072 [stat]. 6

A Lifting Bayes-Optimal Actions to Action Policies

In this section, we formalise the equivalence between the nested optimisation over Bayes-optimal downstream actions in Definition 1 and the joint optimisation over design and action policies used in Equation (5). The key point is that, under standard measurability and integrability assumptions, a pointwise optimisation over actions after observing the experimental history can be lifted to an optimisation over measurable action policies.

Setup. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be the underlying probability space, and $\pi_d \in \Pi_d$ be a fixed design policy. We assume that h_T takes values in a measurable history space $(H, \mathcal{H})$. We denote by ℓ the mapping from pairs $(a, \theta) \in \mathcal{A} \times \Theta$ to its associated loss, $\ell : \mathcal{A} \times \Theta \rightarrow \mathbb{R}$. For a fixed history $h \in H$, we formally define the conditional risk $\forall a \in \mathcal{A}$ as,

$$L(a, h) := \mathbb{E}[\ell(a, \theta) \mid h_T = h].$$

The Bayes-optimal action after observing h is any solution of

$$a^B(h) \in \arg \min_{a \in \mathcal{A}} L(a, h).$$

The corresponding Bayes expected future loss for a design policy π_d is

$$\text{EFL}_\ell^B(\pi_d) = \mathbb{E}_{p(h_T; \pi_d)} \left[\min_{a \in \mathcal{A}} L(a, h_T) \right].$$

We now show that this nested formulation is equivalent to an optimisation over measurable action policies.

Standing assumptions. We work under standard regularity assumptions ensuring that the pointwise Bayes-optimal actions can be selected measurably as functions of the observed history. The history space $(H, \mathcal{H})$ is standard Borel, and the action space $\mathcal{A}$ is a compact metric space with Borel σ -algebra $\mathcal{B}(\mathcal{A})$. For every $\pi_d \in \Pi_d$, the conditional risk $L(a, h)$ admits a jointly measurable version on $\mathcal{A} \times H$, and $a \mapsto L(a, h)$ is lower semicontinuous for every $h \in H$. We also assume the relevant expectations are well-defined, for example through an integrable envelope $m(h_T, \theta)$ satisfying

$$|\ell(a, \theta)| \leq m(h_T, \theta) \quad \text{for all } a \in \mathcal{A}, \quad \mathbb{E}[m(h_T, \theta)] < \infty.$$

Let

$$\Pi_a^{\text{meas}} := \{\pi_a : H \rightarrow \mathcal{A} : \pi_a \text{ is } \mathcal{H}/\mathcal{B}(\mathcal{A})\text{-measurable}\} \quad (11)$$

denote the set of admissible measurable action policies. The restriction to measurable action policies makes the optimisation probabilistically well-defined. Under the regularity assumptions above, the Bayes-action correspondence admits a measurable selector, so the pointwise Bayes action belongs to Π_a^{meas} . It yields the desired equivalence, as stated in the following proposition.

Proposition 1 (Lifting pointwise Bayes actions to action policies). *Under the previous assumptions, for every fixed design policy $\pi_d \in \Pi_d$,*

$$\mathbb{E}_{p(h_T; \pi_d)} \left[\min_{a \in \mathcal{A}} L(a, h_T) \right] = \min_{\pi_a \in \Pi_a^{\text{meas}}} \mathbb{E}_{p(h_T; \pi_d)} [L(\pi_a(h_T), h_T)]. \quad (12)$$

Equivalently,

$$\text{EFL}_\ell^B(\pi_d) = \min_{\pi_a \in \Pi_a^{\text{meas}}} \mathbb{E}_{p(h_T; \pi_d)p(\theta|h_T)} [\ell(\pi_a(h_T), \theta)]. \quad (13)$$

Proof. Fix $\pi_d \in \Pi_d$. For any measurable action policy $\pi_a \in \Pi_a^{\text{meas}}$, we directly have pointwise

$$\min_{a \in \mathcal{A}} L(a, h) \leq L(\pi_a(h), h).$$

Therefore,

$$\mathbb{E}_{p(h_T; \pi_d)} \left[\min_{a \in \mathcal{A}} L(a, h_T) \right] \leq \mathbb{E}_{p(h_T; \pi_d)} [L(\pi_a(h_T), h_T)].$$

Taking the infimum over $\pi_a \in \Pi_a^{\text{meas}}$ gives

$$\mathbb{E}_{p(h_T; \pi_d)} \left[\min_{a \in \mathcal{A}} L(a, h_T) \right] \leq \inf_{\pi_a \in \Pi_a^{\text{meas}}} \mathbb{E}_{p(h_T; \pi_d)} [L(\pi_a(h_T), h_T)].$$

It remains to prove the reverse inequality. Since $L(\cdot, h)$ is lower semicontinuous on the compact action space $\mathcal{A}$, the lower semicontinuous version of Weierstrass' theorem implies that $\arg \min_{a \in \mathcal{A}} L(a, h)$ is non-empty for every $h \in \mathbf{H}$. Hence the argmin correspondence

$$\Gamma(h) := \arg \min_{a \in \mathcal{A}} L(a, h)$$

is non-empty and compact-valued, as a closed subset of $\mathcal{A}$. Since L is jointly measurable and lower semicontinuous in the action variable, the measurable maximum theorem from [1] implies that the value function

$$h \mapsto \min_{a \in \mathcal{A}} L(a, h)$$

is measurable and that Γ is a measurable correspondence. By the Kuratowski–Ryll–Nardzewski measurable selection theorem from [45], there exists a measurable selector $\pi_a^B : \mathbf{H} \rightarrow \mathcal{A}$ such that

$$\pi_a^B(h) \in \Gamma(h) \quad \text{for all } h \in \mathbf{H}.$$

Therefore, $\forall h \in \mathbf{H}$

$$L(\pi_a^B(h), h) = \min_{a \in \mathcal{A}} L(a, h),$$

and hence

$$\mathbb{E}_{p(h_T; \pi_d)} [L(\pi_a^B(h_T), h_T)] = \mathbb{E}_{p(h_T; \pi_d)} \left[\min_{a \in \mathcal{A}} L(a, h_T) \right].$$

Thus the infimum over measurable policies is attained by π_a^B , giving the desired equality. $\square$

From Bayes-action policies to joint design–action optimisation. Having shown the equality (13), we can directly rewrite it as

$$\text{EFL}_\ell^B(\pi_d) = \min_{\pi_a \in \Pi_a^{\text{meas}}} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta; \pi_d)} [\ell(\pi_a(h_T), \theta)]. \quad (14)$$

Consequently, optimising the Bayes expected future loss over design policies can be written as

$$\begin{aligned} \pi_d^B &\in \arg \min_{\pi_d \in \Pi_d} \text{EFL}_\ell^B(\pi_d) \\ &= \arg \min_{\pi_d \in \Pi_d} \min_{\pi_a \in \Pi_a^{\text{meas}}} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta; \pi_d)} [\ell(\pi_a(h_T), \theta)], \end{aligned}$$

which ultimately leads to the core objective used by ACTION-BED as described in equation (5).

Remark. The compactness and lower semicontinuity assumptions above are standard sufficient conditions ensuring that exact Bayes actions exist and admit measurable selectors, which is enough for the equivalence used in this work. When these assumptions fail, the result can be stated in a weaker but more general form using infima and measurable ε -optimal selectors.

Specifically, if for every $\varepsilon > 0$ there exists a measurable policy $\pi_a^\varepsilon : \mathbf{H} \rightarrow \mathcal{A}$ satisfying

$$L(\pi_a^\varepsilon(h), h) \leq \inf_{a \in \mathcal{A}} L(a, h) + \varepsilon,$$

then

$$\mathbb{E}_{p(h_T; \pi_d)} \left[\inf_{a \in \mathcal{A}} L(a, h_T) \right] = \inf_{\pi_a \in \Pi_a^{\text{meas}}} \mathbb{E}_{p(h_T; \pi_d)} [L(\pi_a(h_T), h_T)].$$

This extension is useful when the action space is non-compact or when the conditional risk is not known to attain its pointwise infimum.

B Equivalence Between BED Viewpoints

In the following Section we expand on the discussion on the equivalence between EUR and EFL perspectives of BED by providing a proof of Theorem 1. In section B.2 we additionally elaborate on the equivalence between ACTION-BED and existing variational approaches in a particular case.

B.1 Proof of Theorem 1

We restate the theorem for completeness.

Theorem 1 (Equivalence of EFL & EPU). *Let $s : \mathcal{P}_\theta \times \Theta \rightarrow \mathbb{R}$ be a proper scoring rule on Θ . Then*

$$\pi_d^B = \arg \min_{\pi_d \in \Pi_d} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [s(p_\theta(\cdot | h_T), \theta)] \quad (15)$$

is a Bayes-optimal design policy for the belief model $p(\theta)p(y_{1:T}|\theta;\pi_d)$ and for any loss $\ell \in \mathcal{L}$, where

$$\mathcal{L} = \left\{ \ell : \ell(\tilde{\pi}_a(q_\theta), \theta) = s(q_\theta, \theta), \forall q_\theta \in \mathcal{P}_\theta, \theta \in \Theta \right\}, \quad \tilde{\pi}_a(q_\theta) = \arg \min_{a \in \mathcal{A}} \mathbb{E}_{\theta \sim q_\theta} [\ell(a, \theta)]. \quad (16)$$

Furthermore, the set $\mathcal{L}$ is never empty, and any lower-bounded loss function ℓ induces a corresponding scoring rule s that is proper, though not necessarily strictly proper.

Proof. We start by noting that, for any $\ell \in \mathcal{L}$,

$$\mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [s(p_\theta(\cdot | y_{1:T}; \pi_d), \theta)] = \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [\ell(\tilde{\pi}_a(p_\theta(\cdot | y_{1:T}; \pi_d)), \theta)] \quad (17)$$

$$= \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [\ell(\pi_a^B(y_{1:T}, \pi_d), \theta)] \quad (18)$$

$$= \mathbb{E}_{p(y_{1:T}; \pi_d)} \left[\min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [\ell(a, \theta)] \right]. \quad (19)$$

The first equality follows from the definition of $\mathcal{L}$, while the second follows from the definition of the Bayes action induced by the posterior belief $p_\theta(\cdot | y_{1:T}; \pi_d)$. Therefore, for any $\ell \in \mathcal{L}$, the minimizers of Equation (15) coincide with the Bayes-optimal experimental design policies associated with the downstream loss ℓ , as defined in Definition 1. Hence, π_d^B is a Bayes-optimal experimental design policy for ℓ under the belief model $p(\theta)p(y_{1:T} | \theta; \pi_d)$.

Next, we show that for any ℓ , the corresponding scoring rule induced by Equation (16) is proper. By the definition of s , we have

$$\min_{q_\theta \in \mathcal{P}_\theta} \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [s(q_\theta, \theta)] = \min_{q_\theta \in \mathcal{P}_\theta} \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [\ell(\tilde{\pi}_a(q_\theta), \theta)].$$

Since q_θ influences the expected loss only through the choice of action $\tilde{\pi}_a(q_\theta)$, it follows that

$$\min_{q_\theta \in \mathcal{P}_\theta} \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [\ell(\tilde{\pi}_a(q_\theta), \theta)] \geq \min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [\ell(a, \theta)].$$

Finally, using the definition of $\tilde{\pi}_a$, the right-hand side is attained by choosing the true posterior belief $q_\theta = p_\theta(\cdot | y_{1:T}; \pi_d)$, so that

$$\min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [\ell(a, \theta)] = \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [\ell(\tilde{\pi}_a(p_\theta(\cdot | y_{1:T}; \pi_d)), \theta)]$$

and therefore

$$= \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [s(p_\theta(\cdot | y_{1:T}; \pi_d), \theta)].$$

Thus, $p_\theta(\cdot | y_{1:T}; \pi_d)$ is a minimizer of

$$\mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [s(q_\theta, \theta)],$$

and so s satisfies the definition of a proper scoring rule on Θ . It need not be strictly proper as the minimiser is not necessarily unique.

Finally, we show that $\mathcal{L}$ is never empty. Starting from Equation (15), we have

$$\pi_d^B = \arg \min_{\pi_d \in \Pi_d} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [s(p_\theta(\cdot | y_{1:T}; \pi_d), \theta)] \quad (20)$$

$$= \arg \min_{\pi_d \in \Pi_d} \mathbb{E}_{p(y_{1:T}; \pi_d)} [\mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [s(p_\theta(\cdot | y_{1:T}; \pi_d), \theta)]] \quad (21)$$

Since s is a proper scoring rule, this is equal to

$$\pi_d^B = \arg \min_{\pi_d \in \Pi_d} \mathbb{E}_{p(y_{1:T}; \pi_d)} \left[\min_{q_\theta \in \mathcal{P}_\theta} \mathbb{E}_{p(\theta|y_{1:T}; \pi_d)} [s(q_\theta, \theta)] \right]. \quad (22)$$

This is the general form of a Bayes-optimal experimental design policy with action space $\mathcal{A} = \mathcal{P}_\theta$ and downstream loss

$$\ell(q_\theta, \theta) = s(q_\theta, \theta).$$

Therefore, s is itself an element of $\mathcal{L}$, and consequently $\mathcal{L}$ can never be empty. $\square$

B.2 Recovering Variational Approaches

As noted in Section 3.3, the EIG itself is equivalent, up to a constant offset, to an EPU objective with the log score $s(q_\theta(\cdot), \theta) = -\log q_\theta(\theta)$. That is, up to a constant offset, the EIG is equivalent to

$$\mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [-\log p(\theta|h_T)].$$

We can find an equivalent EFL formulation by choosing $\ell \in \mathcal{L}$ according to Theorem 1. The canonical choice of loss function $\ell \in \mathcal{L}$ for the log score is the log-loss on actions $a(\cdot) \in \mathcal{P}_\theta$, that is $\ell(a(\cdot), \theta) = -\log a(\theta)$. The optimal action under this loss is a standard result:

$$\tilde{\pi}_a = \arg \min_{a(\cdot) \in \mathcal{P}_\theta} \mathbb{E}_{\theta \sim p(\cdot|h_T)} [-\log a(\theta)] = \arg \min_{a(\cdot) \in \mathcal{P}_\theta} \text{KL}(p(\theta|h_T) \| a(\theta)) = p(\theta|h_T),$$

i.e. true Bayesian posterior. Therefore ACTION-BED solves the following joint optimisation problem:

$$(\pi_a, \pi_d)^B = \arg \min_{(\pi_a, \pi_d) \in \Pi_a^{\mathcal{P}_\theta} \times \Pi_d} \text{EFL}_{\log\text{-loss}}(\pi_d, \pi_a), \quad (23)$$

$$\text{where } \text{EFL}_{\log\text{-loss}}(\pi_d, \pi_a) = \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d)} [-\log(\pi_a(h_T), \theta)],$$

and $\Pi_a^{\mathcal{P}_\theta} = \{\pi : \mathcal{Y}^T \times \Xi^T \rightarrow \mathcal{P}_\theta\}$. In practice, we must parameterise $\Pi_a^{\mathcal{P}_\theta}$, or a subset thereof, in order to pose a tractable parametric optimisation problem. We do this by specifying an amortised posterior approximation *policy* $\pi_a^\psi : \mathcal{Y}^T \times \Xi^T \rightarrow \mathcal{P}_\theta$, defined as:

$$\pi_a^\psi(h_T) = q_\psi(\theta|h_T),$$

where $\psi \in \Psi$ parametrises an amortised variational family. Thus, ACTION-BED solves the following parametric optimisation:

$$\min_{\phi, \psi} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d^\phi)} [-\log \pi_a^\psi(h_T)] = \min_{\phi, \psi} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d^\phi)} [-\log q_\psi(\theta|h_T)].$$

This objective is equivalent to the variational posterior objectives introduced by Foster et al. [23, 24], who solve the following problem:

$$\max_{\phi, \psi} \mathbb{E}_{p(\theta)p(y_{1:T}|\theta;\pi_d^\phi)} [\log q_\psi(\theta|h_T) - \log p(\theta)].$$

C Reparameterisation of Simulator and Regularity Conditions

In this Section, we provide details of the reparameterisation function used in our EFL gradient expression Equation (9). We further provide the precise regularity conditions required to differentiate under the integral sign in the reparameterisation trick.

We recall the application of the reparameterisation trick as follows:

$$\nabla_{\phi, \psi} \text{EFL}(\phi, \psi) = \nabla_{\phi, \psi} \mathbb{E}_{p(\theta) p(y_{1:T} | \theta; \pi_d^\phi)} [\ell(\pi_a^\psi(h_T), \theta)] \quad (24)$$

$$= \mathbb{E}_{p(\theta) q(\varepsilon_{1:T})} [\nabla_{\phi, \psi} \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta)], \quad (25)$$

where the simulator $p(y_{1:T} | \theta; \pi_d^\phi)$ was reparameterised in order to write $h_T = h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)$ for $\varepsilon_{1:T} \sim q(\varepsilon_{1:T})$. We firstly provide the exact form of the reparameterisation function in section C.1 before providing regularity conditions on the loss and reparameterisation functions in section C.2 and consequently on the BED model in section C.3.

C.1 Reparametrisation Functions

Recall the data generating process under a given design policy π_d^ϕ is given by $p(y_{1:T} | \theta; \pi_d^\phi) = \prod_{t=1}^T p(y_t | \theta, \xi_t = \pi_d^\phi(h_{t-1}), h_{t-1})$. Furthermore recall the data h_T is defined as $h_T = \{y_{1:T}, \xi_{1:T}\}$ and designs ξ_t are recovered auto-regressively using the design policy and the previous data $\xi_t = \pi_d^\phi(h_{t-1})$.

We can therefore build the reparameterisation of h_T recursively as follows. At the first step, we have $h_1 = \{y_1, \xi_1\}$ where

$$y_1 \sim p(\cdot | \theta, \xi_1), \quad \xi_1 = \pi_d^\phi(\emptyset).$$

Introducing noise variable $\varepsilon_1 \sim q_1(\cdot)$, we can directly write the pair (y_1, ξ_1) as a function of $(\theta, \varepsilon_1; \pi_d^\phi)$, specifically:

$$(y_1, \xi_1) = (\tilde{g}(\theta, \varepsilon_1; \pi_d^\phi), \pi_d^\phi(\emptyset)) =: h_1(\theta, \varepsilon_1; \pi_d^\phi).$$

At the second time step, we now have

$$y_2 \sim p(\cdot | \theta, \xi_2, h_1), \quad \xi_2 = \pi_d^\phi(h_1).$$

Again introducing noise variable $\varepsilon_2 \sim q_2(\cdot)$, we can express the pair (y_2, ξ_2) as a function of $(\theta, \varepsilon_2, h_1; \pi_d^\phi)$ as follows:

$$(y_2, \xi_2) = (\tilde{g}(\theta, \varepsilon_2, h_1; \pi_d^\phi), \pi_d^\phi(h_1)) =: \tilde{h}_2(\theta, \varepsilon_2, h_1; \pi_d^\phi)$$

By substituting the reparameterisation of the previous step, i.e. $h_1 = h_1(\theta, \varepsilon_1; \pi_d^\phi)$ for $\varepsilon_1 \sim q_1(\cdot)$ and stacking with h_1 , we obtain:

$$h_2 = (y_1, \xi_1, y_2, \xi_2) = (h_1, y_2, \xi_2) = (h_1, \tilde{h}_2) =: h_2(\theta, \varepsilon_{1:2}; \pi_d^\phi).$$

We continue this argument recursively, arriving at a reparameterisation function for the entire experimental history $h_T = h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)$ for $\varepsilon_{1:T} \sim \prod_{t=1}^T q_t(\cdot)$. Note that we may sometimes abuse notation slightly and write this as $h_T = h_T(\theta, \varepsilon_{1:T}, \phi)$, with the dependence on the policy π_d implicit.

C.2 Conditions for Differentiation Under the Integral Sign

Let $(\Omega, \mathcal{F}, \mu)$ be a measure space on which the random variables θ and ε are defined. Let $\Phi \times \Psi \subset \mathbb{R}^{d_\phi} \times \mathbb{R}^{d_\psi}$ be an open set. Equation 25 is justified under the following conditions:

1. The distributions $p(\theta)$ and $p(\varepsilon_{1:T})$ are independent of (ϕ, ψ) ;
2. For μ -almost every $(\theta, \varepsilon_{1:T}) \in \Omega$, the mapping

$$(\phi, \psi) \mapsto \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta)$$

is $\mathcal{F}$ -measurable and integrable:

$$\int_{\Omega} \left| \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta) \right| d\mu(\theta, \varepsilon_{1:T}) < \infty.$$

3. For μ -almost every $(\theta, \varepsilon_{1:T}) \in \Omega$, the mapping is differentiable in (ϕ, ψ) and the partial derivatives

$$\partial_\phi \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta), \quad \partial_\psi \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta)$$

exist (which holds under standard smoothness assumptions on h_T , π_a^ψ , and ℓ);

4. For each compact $K \subset \Phi \times \Psi$, there exist integrable functions

$$g_\phi : \Omega \rightarrow [0, \infty), \quad g_\psi : \Omega \rightarrow [0, \infty)$$

such that for all $(\phi, \psi) \in K$ and μ -almost every $(\theta, \varepsilon_{1:T}) \in \Omega$,

$$|\partial_\phi \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta)| \leq g_\phi(\omega), \quad \text{and} \quad \int_\Omega g_\phi(\theta, \varepsilon_{1:T}) d\mu(\theta, \varepsilon_{1:T}) < \infty, \quad (26)$$

and

$$|\partial_\psi \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta)| \leq g_\psi(\omega), \quad \text{and} \quad \int_\Omega g_\psi(\theta, \varepsilon_{1:T}) d\mu(\theta, \varepsilon_{1:T}) < \infty. \quad (27)$$

These conditions follow from the differentiation-under-the-integral theorem. The following section explains how these necessary conditions for differentiating under the integral sign can be derived from assumptions on the model.

C.3 Conditions on the BED model

Let $\Phi \subset \mathbb{R}^{d_\phi}$ and $\Psi \subset \mathbb{R}^{d_\psi}$ be open parameter sets. In order to have the map

$$(\phi, \psi) \mapsto \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta)$$

differentiable, we have to satisfy the following conditions:

1. **Differentiable design network:** The design-induced history

$$h_T : \Theta \times \mathcal{E} \times \Phi \rightarrow \mathcal{Y}_T \times \Xi^T$$

$$(\theta, \varepsilon_{1:T}, \phi) \mapsto h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi),$$

is measurable in $(\theta, \varepsilon_{1:T})$ and differentiable in ϕ for almost all $(\theta, \varepsilon_{1:T})$, with

$$\nabla_\phi h_T(\theta, \varepsilon_{1:T}, \phi)$$

existing and being finite. In particular, in all deep learning architectures considered to model h_T , this is implemented as a composition of differentiable operations with respect to ϕ (e.g. affine maps and smooth or piecewise-smooth activations), so that it is differentiable almost everywhere.

2. **Differentiable prediction network:** The downstream prediction network

$$\pi_a^\psi : \mathcal{H}_T \rightarrow \Theta$$

is measurable in its input and differentiable in (ψ, h_T) , i.e.

$$(\psi, h) \mapsto \pi_a^\psi(h)$$

is differentiable for all (ψ, h) , or at least almost everywhere, with Jacobians

$$\nabla_\psi \pi_a^\psi(h), \quad \nabla_h \pi_a^\psi(h)$$

existing and finite. This holds in particular when π_a^ψ is a neural network with standard smooth (or piecewise-linear) activations such as tanh, softplus, ReLU or leaky-ReLU, which are differentiable almost everywhere.

3. **Differentiable loss:** The downstream loss

$$\ell : \Theta \times \Theta \rightarrow \mathbb{R}, \quad (a, \theta) \mapsto \ell(a, \theta),$$

is differentiable in its first argument a for all (a, θ) (or almost everywhere), with gradient

$$\nabla_a \ell(a, \theta)$$

existing and finite (e.g. mean-squared error, cross-entropy, etc.).

Under these conditions, for almost all $(\theta, \varepsilon_{1:T})$, the composite map

$$(\phi, w) \mapsto \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}, \phi)), \theta)$$

is differentiable, and its gradient can be computed by the chain rule.

D Gradient Computations

D.1 Gradient Computations for Separated Networks

We denote, for a single draw $(\theta, \varepsilon_{1:T}) \sim p(\theta)q(\varepsilon_{1:T})$,

$$L(\phi, \psi; \theta, \varepsilon_{1:T}) := \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta),$$

so that

$$\widehat{\nabla_{\phi, \psi} \text{EFL}}(\phi, \psi) = \nabla_{\phi, \psi} L(\phi, \psi; \theta, \varepsilon_{1:T}). \quad (28)$$

Gradient with respect to ψ . Since h_T does not depend on ψ , we have

$$\nabla_\psi L(\phi, \psi; \theta, \varepsilon_{1:T}) = \nabla_\psi \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta) \quad (29)$$

$$= \nabla_a \ell(a, \theta) \Big|_{a=\pi_a^\psi(h_T)} \nabla_\psi \pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \quad (30)$$

where $\nabla_a \ell(a, \theta)$ denotes the gradient of the loss with respect to its first argument.

Gradient with respect to ϕ . Here the dependence on ϕ is only through the history $h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)$, so the chain rule yields

$$\nabla_\phi L(\phi, \psi; \theta, \varepsilon_{1:T}) = \nabla_\phi \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)), \theta) \quad (31)$$

$$= \nabla_a \ell(a, \theta) \Big|_{a=\pi_a^\psi(h_T)} \nabla_{h_T} \pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi)) \nabla_\phi h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi). \quad (32)$$

Joint gradient estimator. Stacking the two components, a single-sample Monte Carlo estimator of the full gradient is thus

$$\widehat{\nabla_{\phi, \psi} \text{EFL}}(\phi, \psi) = \begin{pmatrix} \nabla_\phi L(\phi, \psi; \theta, \varepsilon_{1:T}) \\ \nabla_\psi L(\phi, \psi; \theta, \varepsilon_{1:T}) \end{pmatrix}.$$

In practice, automatic differentiation computes precisely these quantities by backpropagating through the computational graph

$$(\phi, \psi, \theta, \varepsilon_{1:T}) \mapsto h_T(\theta, \varepsilon_{1:T}; \pi_d^\phi) \mapsto \pi_a^\psi(h_T) \mapsto \ell(\pi_a^\psi(h_T), \theta).$$

D.2 Gradient Estimator in case of weight sharing

We now make explicit the form of the Monte Carlo gradient estimator in the case where both the acquisition and downstream policies share the same encoder network, with a sum-pooling aggregation step of the encodings and a specific prediction head for both. It is a common choice of architecture for policy-based methods. For a single draw $(\theta, \varepsilon_{1:T}) \sim p(\theta)q(\varepsilon_{1:T})$, we consider the loss

$$L(\phi_1, \phi_2, \psi; \theta, \varepsilon_{1:T}) := \ell(\pi_a^\psi(h_T(\theta, \varepsilon_{1:T}; \pi_d^{\phi_1, \phi_2})), \theta),$$

where the downstream prediction network is defined via a shared DeepSets-style encoder:

$$e_t = E_{\phi_1}(\xi_t, y_t) \in \mathbb{R}^d, \quad t = 1, \dots, T, \quad (33)$$

$$R(h_T) = \sum_{t=1}^T e_t = \sum_{t=1}^T E_{\phi_1}(\xi_t, y_t), \quad (34)$$

$$\pi_a^\psi(h_T) = f_\psi(R(h_T)). \quad (35)$$

A single-sample Monte Carlo estimator of the gradient of the expected future loss is then

$$\widehat{\nabla_{\phi_1, \phi_2, \psi} \text{EFL}}(\phi_1, \phi_2, \psi) = \nabla_{\phi_1, \phi_2, \psi} L(\phi_1, \phi_2, \psi; \theta, \varepsilon_{1:T}).$$

Gradient with respect to the prediction head ψ . The dependence on ψ is only through $a = f_\psi(R(h_T))$, so the chain rule yields

$$\nabla_\psi L(\phi_1, \phi_2, \psi; \theta, \varepsilon_{1:T}) = \nabla_\psi \ell(f_\psi(R(h_T)), \theta) \quad (36)$$

$$= \nabla_a \ell(a, \theta) \Big|_{a=f_\psi(R(h_T))} \nabla_\psi f_\psi(R(h_T)). \quad (37)$$

Gradient with respect to the shared encoder parameters ϕ_1 . Here, ϕ_1 affects the loss both through the encoder outputs $e_t = E_{\phi_1}(\xi_t, y_t)$ and, through the design policy, through the generated history $h_T(\theta, \varepsilon_{1:T}; \pi_d^{\phi_1, \phi_2})$. Applying the chain rule gives

$$\nabla_{\phi_1} L(\phi_1, \phi_2, \psi; \theta, \varepsilon_{1:T}) = \nabla_a \ell(a, \theta) \Big|_{a=f_\psi(R(h_T))} \nabla_R f_\psi(R(h_T)) \nabla_{\phi_1} R(h_T). \quad (38)$$

The term $\nabla_{\phi_1} R(h_T)$ includes both the direct dependence of the encoder E_{ϕ_1} on ϕ_1 and the indirect dependence through the history $h_T(\theta, \varepsilon_{1:T}; \pi_d^{\phi_1, \phi_2})$. In particular,

$$\nabla_{\phi_1} R(h_T) = \nabla_{\phi_1} \left[\sum_{t=1}^T E_{\phi_1}(\xi_t, y_t) \right], \quad (39)$$

where $(\xi_t, y_t)_{t=1}^T$ are generated by the design policy $\pi_d^{\phi_1, \phi_2}$.

Gradient with respect to the design head ϕ_2 . The parameters ϕ_2 enter through the design policy that generates the history $h_T(\theta, \varepsilon_{1:T}; \pi_d^{\phi_1, \phi_2})$. Denoting this dependence explicitly, the chain rule gives

$$\nabla_{\phi_2} L(\phi_1, \phi_2, \psi; \theta, \varepsilon_{1:T}) = \nabla_{h_T} \ell(f_\psi(R(h_T)), \theta) \nabla_{\phi_2} h_T(\theta, \varepsilon_{1:T}; \pi_d^{\phi_1, \phi_2}), \quad (40)$$

where the term $\nabla_{h_T} \ell(\cdot)$ itself decomposes through the encoder and pooling:

$$\nabla_{h_T} \ell(f_\psi(R(h_T)), \theta) = \nabla_a \ell(a, \theta) \Big|_{a=f_\psi(R(h_T))} \nabla_R f_\psi(R(h_T)) \nabla_{h_T} R(h_T). \quad (41)$$

In practice, automatic differentiation computes exactly these gradients by backpropagating through the computational graph

$$(\phi_1, \phi_2, \psi, \theta, \varepsilon_{1:T}) \mapsto h_T(\theta, \varepsilon_{1:T}; \pi_d^{\phi_1, \phi_2}) \mapsto \{E_{\phi_1}(\xi_t, y_t)\}_{t=1}^T \mapsto R(h_T) \mapsto f_\psi(R(h_T)) \mapsto \ell(\cdot, \theta).$$

Finally, a Monte Carlo estimator of the full gradient is obtained by averaging these single-sample gradients over i.i.d. draws $(\theta^{(i)}, \varepsilon_{1:T}^{(i)})$.

E Network Architecture

Our co-optimization framework, introduced in Section 4, is agnostic to the specific parameterization of the design policy π_d^ϕ and the downstream action policy π_a^ψ . Any differentiable architecture can be used, provided that the resulting computation graph supports pathwise optimization through the reparameterized simulator. Nevertheless, architectural choices play an important role in practice, since they determine how the experimental history is represented and which inductive biases are made available to the policy. In particular, the architecture can encode exchangeability, temporal ordering, Markovian structure, spatial symmetries, or more general dependencies between past designs and outcomes.

Design policy. The design policy π_d^ϕ is the central component of the acquisition mechanism. At each experimental step, it maps the current history

$$h_t = \{(\xi_s, y_s)\}_{s=1}^t$$

to the next design, or to a distribution over the designs,

$$\begin{cases} \xi_{t+1} = \pi_d^\phi(h_t) \\ \xi_{t+1} \sim p_\phi(\xi | h_t) =: \pi_d^\phi(h_t) \end{cases}$$

In deep adaptive design, this mapping is commonly parameterized through an encoder–aggregator–emitter architecture. First, each observed design–outcome pair (ξ_s, y_s) is embedded by a local encoder,

$$r_s = E_{\phi_1}(\xi_s, y_s),$$

or, more generally, by separate design and outcome encoders followed by a fusion module. The resulting sequence or set of pair embeddings $(r_1, \dots, r_t)$ is then compressed into a global history representation,

$$z_t = A_{\phi_2}(r_1, \dots, r_t),$$

where A_{ϕ_2} is a history aggregation module. Finally, an emitter network maps this history state to the next design,

$$\xi_{t+1} = G_{\phi_3}(z_t),$$

or to a distribution over candidate designs in stochastic acquisition policies.

The main decision-making capacity of the design policy is often concentrated in the aggregation module. The encoder produces local representations of individual experiments, whereas the aggregator determines how these pieces of evidence are combined to support the next acquisition decision. Different choices of A_{ϕ_2} encode different assumptions about the structure of the history. For exchangeable probabilistic models of histories, permutation-invariant architectures such as DeepSets [22, 67] are natural: they aggregate pair embeddings through a symmetric pooling operation and therefore make the design decision independent of the ordering of observations. For temporally ordered or dynamically evolving systems, recurrent architectures such as LSTMs [29, 39] can encode the sequential dependence between past interventions and outcomes. Attention-based architectures, including Set Transformers [47] and Transformer Neural Process (TNP)-style encoders [3], provide a more flexible alternative by allowing the policy to learn interactions between all past design–outcome pairs and to condition the next design on the most informative parts of the history.

Downstream action network. The downstream action policy maps the final experimental history to a terminal action,

$$a_T = \pi_a^\psi(h_T).$$

This action may represent a point estimate, a classifier output, a decision rule, or any task-specific prediction required after data acquisition. Since the downstream policy is only evaluated after the acquisition phase and needs to extract information contained in the history, π_a^ψ can be implemented with a lighter architecture, or can follow the same encoder–aggregator–emitter structure as the design policy:

$$r_s = E_{\psi_1}(\xi_s, y_s), \quad z_T = A_{\psi_2}(r_1, \dots, r_T), \quad a_T = G_{\psi_3}(z_T).$$

This shared architectural template is especially useful when the downstream action depends on subtle interactions across the entire history. In such cases, using a rich history encoder for π_a^ψ can improve the quality of the terminal decision and provide stronger gradients to guide the design policy during co-optimization.

Enhancing coupling through weight sharing. When the design policy and downstream action policy are built from compatible encoder–aggregator architectures, one can further strengthen their coupling by sharing part of their representations. For example, both policies may use a common local encoder E_ω for design–outcome pairs, while retaining separate aggregation and output heads:

$$r_s = E_\omega(\xi_s, y_s),$$

$$\xi_{t+1} = G_d^\phi \left(A_d^\phi(r_1, \dots, r_t) \right), \quad a_T = G_a^\psi \left(A_a^\psi(r_1, \dots, r_T) \right).$$

A stronger form of sharing can also be used, where the local encoder and part of the history aggregator are shared, while task-specific heads remain distinct:

$$z_t = A_\omega(E_\omega(\xi_1, y_1), \dots, E_\omega(\xi_t, y_t)),$$

$$\xi_{t+1} = G_d^\phi(z_t), \quad a_T = G_a^\psi(z_T).$$

This form of weight sharing has several advantages. First, it encourages both the design policy and the downstream predictor to build compatible representations of the experimental history. The design policy is then trained to collect data that are useful under the same representational geometry used by the downstream action network. Second, sharing parameters strengthens the coupling between the two objectives: gradients from the downstream loss directly shape the representation used for acquisition. Third, it reduces the number of free parameters, which can improve statistical efficiency and regularize training, especially when the number of simulated trajectories is limited.

E.1 TNP-Like Policy Architecture

In the experiments, we use a modular neural architecture for the adaptive design policy π_d^ϕ . The architecture follows the general encoder–aggregator–emitter decomposition described above, with a Transformer-based module used to summarize the experimental history. Its purpose is to map the information collected so far into the next design decision, while remaining flexible enough to handle different experimental tasks. The architecture can be viewed as a TNP-like design policy where past design–outcome pairs play the role of context observations, and a learned decision token queries this context in order to produce the next design. We detail below the different modules composing the architecture used throughout the experiments. Experiment-specific dimensions and hyperparameters are denoted by variables, whereas numerical values indicate settings shared across all experiments.

Encoding design–outcome pairs. Each past design ξ_t and outcome y_t is first mapped to a latent representation. In practice, the design and outcome are encoded separately,

$$e_t^\xi = f_\xi(\xi_t), \quad e_t^y = f_y(y_t),$$

Table 5: Design encoder architecture.

Layer	Description	Dimension	Activation
Input	ξ	$\dim_\xi$	–
H1	Fully connected	64	ReLU
Output	Fully connected	$d_{\text{enc},\xi}$	–

Table 6: Outcome encoder architecture.

Layer	Description	Dimension	Activation
Input	y	$\dim_y$	–
H1	Fully connected	64	ReLU
Output	Fully connected	$d_{\text{enc},y}$	–

and then combined into a single representation of the experiment performed at step t :

$$r_t = f_{\text{pair}} \left([e_t^\xi; e_t^y] \right).$$

Table 7: Fusion module architecture.

Layer	Description	Dimension	Activation
Input	$[e^\xi; e^y]$	$d_{\text{enc},y} + d_{\text{enc},\xi}$	–
H1	Fully connected	64	GELU
Output	Fully connected	$d_{\text{enc},r}$	–

This pair representation captures the local information revealed by one design–outcome interaction. Applying the same construction to all previous steps produces a sequence of history tokens

$$R_t = (r_1, \dots, r_t).$$

When the task is temporally structured, these tokens can also be augmented with time information, so that the policy can distinguish early and late observations. When the task is exchangeable, the same representation can instead be interpreted as an unordered set of observed experiments.

Aggregating the experimental history. The sequence of pairwise representations $(r_1, \dots, r_t)$ is processed by a Transformer-based history aggregator. Each representation is first projected to a common embedding dimension, then enriched with a learned type embedding indicating that it corresponds to a history token. When time embeddings are enabled, a learned temporal embedding is also added to preserve the order of the acquisition trajectory. A learned decision token is appended at the end of the sequence and is assigned its own type embedding, together with the time index corresponding to the next decision step. The resulting token sequence is normalized and passed through a stack of Transformer encoder layers.

The final representation of the decision token is used as the summary of the experimental history. This token acts as a learnable query that attends to all previous design–outcome pairs and extracts the information relevant for choosing the next design. Denoting this representation by h_t , the aggregation step can be written as

$$h_t = f_{\text{hist}}(r_1, \dots, r_t), \quad h_t \in \mathbb{R}^{d_h}.$$

This attention-based aggregation avoids imposing a fixed pooling rule over the experimental history. It allows the policy to model interactions between past observations, account for the temporal order of the acquisitions, and focus on the parts of the trajectory that are most informative for the next experimental decision.

Table 8: History aggregator architecture. The module maps the sequence of pair representations $(r_1, \dots, r_t)$ to a single history representation h_t .

Component	Configuration	Output shape
Input sequence	Pair representations $(r_1, \dots, r_t), r_i \in \mathbb{R}^{d_{\text{enc},r}}$	$t \times d_{\text{enc},r}$
Input adapter	Linear layer $d_{\text{enc},r} \rightarrow d_{\text{enc},h}$	$t \times d_{\text{enc},h}$
History token embeddings	Add learned history type embeddings	$t \times d_{\text{enc},h}$
Time embeddings	Add learned time embeddings, enabled	$t \times d_{\text{enc},h}$
Decision token	Append learned query token with decision type embedding	$(t+1) \times d_{\text{enc},h}$
Transformer encoder	2 layers, 4 heads, feedforward dimension 128	$(t+1) \times d_{\text{enc},h}$
Readout	Final decision-token representation	$d_{\text{enc},h}$

Emitting the next design. Finally, an emitter network maps the history representation to the next design:

$$\xi_{t+1} = f_{\text{emit}}(h_t).$$

Table 9: Emitter network architecture.

Layer	Description	Dimension	Activation
Input	History representation h_t	$d_{\text{enc},h}$	–
H1	Fully connected	64	GELU
Output	Fully connected	$\dim_{\xi}$	–

Complete and modular design policy. The complete design policy is obtained by composing the modules. Given the experimental history $\mathcal{H}_t = \{(\xi_i, y_i)\}_{i=1}^t$, the policy can be written as

$$\pi_{\phi}(\mathcal{H}_t) = f_{\text{emit}} \circ f_{\text{hist}} \circ f_{\text{pair}} \circ (f_{\xi}, f_y)(\{(\xi_i, y_i)\}_{i=1}^t).$$

A key advantage of this architecture is its modularity. The task-specific encoders f_{ξ} and f_y can be adapted to the structure of each experiment, while the fusion, history aggregation, and emission

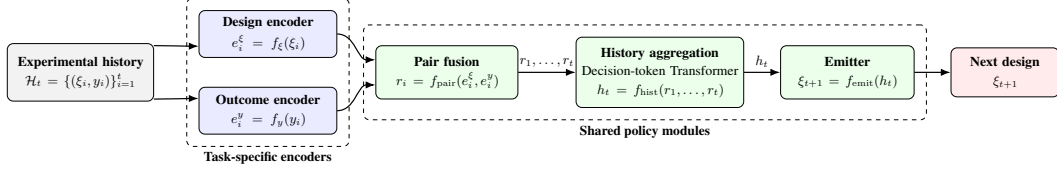


Figure 3: Modular architecture of the design policy. Past designs and outcomes are encoded separately, fused into pair representations, aggregated by a decision-token Transformer, and mapped by the emitter to the next design.

modules are kept fixed. In low-dimensional settings, the encoders are simple multilayer perceptrons, whereas for structured or high-dimensional observations, f_y can be replaced by a convolutional or patch-based encoder. This architecture therefore provides a common backbone across all experiments, both for our task-driven method and for the DAD/iDAD baselines. The same policy network can be trained end-to-end by backpropagating gradients either through the downstream task loss or through the sequential PCE objective.

E.2 Downstream warm-up

In practice, before running the joint optimisation described in algorithm 1, we may initialise the action policy with a short downstream warm-up phase. During this phase, the design policy is kept fixed and is not used to generate designs. Instead, design trajectories are sampled from a task-relevant random distribution $r(\xi_{1:T})$, and only the downstream-action policy π_a^ψ is trained to minimise the target downstream loss. Given samples $\theta^b \sim p(\theta)$, simulator noise $\varepsilon_{1:T}^b \sim q(\varepsilon_{1:T})$, and random designs $\xi_{1:T}^b \sim r(\xi_{1:T})$, the warm-up gradient estimator is

$$\widehat{\nabla}_\psi \mathcal{L}_{\text{down}}(\psi) = \frac{1}{B} \sum_{b=1}^B \nabla_\psi \ell(\pi_a^\psi(h_T(\theta^b, \varepsilon_{1:T}^b; \xi_{1:T}^b)), \theta^b). \quad (42)$$

Although the resulting random designs are generally suboptimal, they provide an initial supervised signal for the action policy. Empirically, this warm-up can improve the subsequent joint optimisation: when the design policy starts being trained, the downstream network already maps histories to meaningful actions, yielding more informative gradients for the design policy and leading to a better coupling between acquisition and downstream prediction.

Algorithm 2 ACTION-BED with downstream warm-up

Require: Policies π_d^ϕ, π_a^ψ , initialisations ϕ_0, ψ_0
Require: Warm-up steps K_{warm} , joint steps K , random design distribution $r(\xi_{1:T})$

Downstream warm-up

- 1: **for** $k = 0, 1, \dots, K_{\text{warm}} - 1$ **do**
- 2: $\theta^{1:B}, \varepsilon_{1:T}^{1:B}, \xi_{1:T}^{1:B} \sim p(\theta)q(\varepsilon_{1:T})r(\xi_{1:T})$
- 3: $g_k^\psi \leftarrow \widehat{\nabla}_\psi \mathcal{L}_{\text{down}}(\psi_k)$ (eq. 42)
- 4: $\psi_{k+1} \leftarrow \text{UPDATE}_a(\psi_k, g_k^\psi)$
- 5: **end for**
- 6: $\psi_0 \leftarrow \psi_{K_{\text{warm}}}$

Joint Action-BED optimisation

- 7: **for** $k = 0, 1, \dots, K - 1$ **do**
- 8: $\theta^{1:B}, \varepsilon_{1:T}^{1:B} \sim p(\theta)q(\varepsilon_{1:T})$
- 9: $\phi_k, g_k^\psi \leftarrow \widehat{\nabla}_{\phi, \psi} \text{EFL}(\phi_k, \psi_k)$ (eq. 10)
- 10: $\phi_{k+1}, \psi_{k+1} \leftarrow \text{UPDATE}(\phi_k, g_k^\phi, \psi_k, g_k^\psi)$
- 11: **end for**
- 12: **return** ϕ_K, ψ_K

F Bayes-optimal action under MSE and permutation-invariant MSE

We provide here a self-contained derivation of the Bayes-optimal action associated with the downstream decision problem considered in the source location finding experiment in Section 6.1. Since the general Bayes-action formulation is introduced in the main text, we only recall the minimal setup needed for the derivation. We then derive the posterior-mean optimality result for the standard squared error loss and extend it to the permutation-invariant MSE used in the experiment.

F.1 Bayes-action setup

Let $\theta \in \Theta$ denote the latent parameter and let $h_T \in \mathcal{H}_T$ denote the experimental trajectory induced by a fixed data-acquisition policy π_a^ϕ . A downstream action policy is a measurable function

$$\pi_a^\psi : \mathcal{H}_T \rightarrow \mathcal{A},$$

implemented in practice by the downstream-action network.

For a loss function $\ell : \mathcal{A} \times \Theta \rightarrow \mathbb{R}_+$, and a fixed trajectory h_T , we call the Bayes-optimal action the minimizer of the posterior expected loss:

$$a^*(h_T) = \arg \min_{a \in \mathcal{A}} \mathbb{E}_{p(\theta|h_T)} [\ell(a, \theta)]. \quad (43)$$

F.2 Standard MSE loss

We first specialize to the standard squared error loss. We assume that $\Theta = \mathcal{A} \subseteq \mathbb{R}^d$, which is the case for the source location finding task, and we define

$$\ell_{\text{MSE}}(\hat{\theta}, \theta) = \|\hat{\theta} - \theta\|_2^2.$$

For a fixed trajectory h_T , the conditional risk is

$$\mathcal{R}_{\text{MSE}}(\hat{\theta} \mid h_T) = \mathbb{E}_{p(\theta|h_T)} [\|\hat{\theta} - \theta\|_2^2]. \quad (44)$$

Expanding the squared norm gives

$$\begin{aligned} \mathcal{R}_{\text{MSE}}(\hat{\theta} \mid h_T) &= \mathbb{E} [\|\hat{\theta}\|_2^2 - 2\langle \hat{\theta}, \theta \rangle + \|\theta\|_2^2 \mid h_T] \\ &= \|\hat{\theta}\|_2^2 - 2\langle \hat{\theta}, \mathbb{E}[\theta \mid h_T] \rangle + \mathbb{E} [\|\theta\|_2^2 \mid h_T]. \end{aligned} \quad (45)$$

The last term does not depend on $\hat{\theta}$. Differentiating with respect to $\hat{\theta}$ and setting the gradient to zero yields

$$\nabla_{\hat{\theta}} \mathcal{R}_{\text{MSE}}(\hat{\theta} \mid h_T) = 2\hat{\theta} - 2\mathbb{E}[\theta \mid h_T] = 0.$$

Hence,

$$\hat{\theta}_{\text{MSE}}^*(h_T) = \mathbb{E}[\theta \mid h_T]. \quad (46)$$

Thus, under the standard MSE loss, the Bayes-optimal action is the posterior mean.

F.3 Permutation-invariant MSE loss

However, in our setting the latent parameter is not naturally ordered. For example, in the source location finding problem with K exchangeable sources, the parameter can be written as

$$\theta = (\theta_1, \dots, \theta_K) \in \mathbb{R}^{K \times d},$$

but the ordering of the sources is arbitrary. In that case, penalizing an estimate $\hat{\theta}$ using the standard MSE would incorrectly distinguish between two estimates that differ only by a permutation of the sources. Let $\mathcal{S}_K$ denote the symmetric group over K elements. For a permutation $\sigma \in \mathcal{S}_K$, write

$$\sigma \cdot \theta = (\theta_{\sigma(1)}, \dots, \theta_{\sigma(K)}).$$

The permutation-invariant MSE is defined as

$$\ell_{\text{PI-MSE}}(\hat{\theta}, \theta) = \min_{\sigma \in \mathcal{S}_K} \frac{1}{K} \sum_{k=1}^K \|\hat{\theta}_k - \theta_{\sigma(k)}\|_2^2. \quad (47)$$

Equivalently, identifying permutations with permutation matrices $P \in \mathcal{P}_K$, this can be written as

$$\ell_{\text{PI-MSE}}(\hat{\theta}, \theta) = \frac{1}{K} \min_{P \in \mathcal{P}_K} \left\| \hat{\theta} - P\theta \right\|_F^2, \quad (48)$$

where $\|\cdot\|_F$ denotes the Frobenius norm.

For a fixed trajectory h_T , the Bayes-optimal action under this loss is therefore

$$\hat{\theta}_{\text{PI-MSE}}^*(h_T) = \arg \min_{\hat{\theta} \in \mathbb{R}^{K \times d}} \mathbb{E}_{p(\theta|h_T)} \left[\frac{1}{K} \min_{P \in \mathcal{P}_K} \left\| \hat{\theta} - P\theta \right\|_F^2 \right]. \quad (49)$$

This is the Fréchet mean of the posterior distribution on the quotient space $\mathbb{R}^{K \times d} / \mathcal{S}_K$, where configurations that differ only by a permutation are identified. To make the optimality condition explicit, define, for a given estimate $\hat{\theta}$ and latent parameter θ , an optimal alignment

$$P_{\hat{\theta}}(\theta) \in \arg \min_{P \in \mathcal{P}_K} \left\| \hat{\theta} - P\theta \right\|_F^2. \quad (50)$$

Assuming that this optimal alignment is unique almost surely under $p(\theta | h_T)$, the conditional risk can be written locally as

$$\mathcal{R}_{\text{PI-MSE}}(\hat{\theta} | h_T) = \frac{1}{K} \mathbb{E}_{p(\theta|h_T)} \left[\left\| \hat{\theta} - P_{\hat{\theta}}(\theta)\theta \right\|_F^2 \right]. \quad (51)$$

By the envelope theorem, differentiating the minimized objective with respect to $\hat{\theta}$ gives

$$\begin{aligned} \nabla_{\hat{\theta}} \mathcal{R}_{\text{PI-MSE}}(\hat{\theta} | h_T) &= \frac{2}{K} \mathbb{E}_{p(\theta|h_T)} \left[\hat{\theta} - P_{\hat{\theta}}(\theta)\theta \right] \\ &= \frac{2}{K} \left(\hat{\theta} - \mathbb{E}_{p(\theta|h_T)} [P_{\hat{\theta}}(\theta)\theta] \right). \end{aligned} \quad (52)$$

Therefore, any differentiable Bayes-optimal action satisfies the fixed-point condition

$$\hat{\theta}_{\text{PI-MSE}}^*(h_T) = \mathbb{E}_{p(\theta|h_T)} \left[P_{\hat{\theta}_{\text{PI-MSE}}^*(h_T)}(\theta)\theta \right]. \quad (53)$$

Equation (53) shows that the Bayes-optimal action under permutation-invariant MSE is the posterior mean after optimally aligning each posterior sample to the estimate itself. The Bayes-optimal action is also not unique in the usual Euclidean sense: if $\hat{\theta}^*$ is optimal, then $P\hat{\theta}^*$ is also optimal for any permutation matrix $P \in \mathcal{P}_K$. Hence, the estimator is identifiable only up to permutation, which is precisely the desired invariance property.

F.4 Interpretation for the downstream network

Under the standard MSE loss, minimizing the empirical Bayesian risk trains the downstream-action network to approximate the posterior mean mapping

$$h_T \mapsto \mathbb{E}[\theta | h_T].$$

Under the permutation-invariant MSE, the downstream network instead learns a representative of the posterior Fréchet mean on the quotient space induced by the permutation symmetry:

$$h_T \mapsto \arg \min_{\hat{\theta}} \mathbb{E}_{p(\theta|h_T)} \left[\min_{P \in \mathcal{P}_K} \frac{1}{K} \left\| \hat{\theta} - P\theta \right\|_F^2 \right].$$

Equivalently, it learns an unordered set-valued estimate of the latent sources. During training and evaluation, the optimal matching in Equation (47) removes the arbitrary label ordering of the sources, ensuring that two estimates that differ only by a permutation incur the same loss.

G Experiment Details

The implementation was carried out in part using the PyTorch library [56], and the code is publicly available. Experiments were conducted on a server equipped with an NVIDIA Titan RTX GPU (24GB, Turing architecture) and 24 CPU cores. Each experiment was run on a single GPU.

G.1 Source Location Finding

The *source location finding* problem is a canonical benchmark in BED, in which the objective is to infer the unknown spatial positions of K signal-emitting sources $\theta_1, \dots, \theta_K \in \mathbb{R}^2$ from a sequence of noisy measurements. Following the experimental setting of [22], at each design step t , an agent selects a sensor location $\xi_t \in \mathbb{R}^2$, and observes a scalar response $y_t \in \mathbb{R}$ corresponding to the aggregation of the signals emitted by the K sources, whose expected magnitudes decay with the squared distance to the sensor according to a known physical attenuation model. A commonly used formulation assumes

$$y_t = b + \sum_{k=1}^K \frac{A}{m + \|\xi_t - \theta_k\|^2} + \sigma \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1), \quad (54)$$

where $A > 0$ denotes the signal amplitude, b is a background offset, $m > 0$ is a stabilising constant preventing singularities at zero distance, and σ controls the measurement noise level. Equivalently, denoting the noiseless aggregated intensity by

$$\mu(\theta, \xi) = b + \sum_{k=1}^K \frac{A}{m + \|\xi - \theta_k\|^2},$$

the observation model can be written in terms of noisy measurements of the log total intensity, yielding the prior–likelihood specification

$$\theta_k \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0_2, I_2), \quad \log y \mid \theta, \xi \sim \mathcal{N}(\log \mu(\theta, \xi), \sigma),$$

where $\theta = (\theta_1, \dots, \theta_K) \in \mathbb{R}^{K \times 2}$. This formulation clearly shows that the model satisfies the reparameterization assumption C.1. The goal is to localize the sources with the highest possible accuracy: we seek to collect data from which a downstream model can be trained to identify all sources as effectively as possible. In our experimental setting, we consider the case $K = 2$.

The task is challenging because information is spatially structured and highly non-uniform: sensor placements far from the source yield almost uninformative responses, while overly greedy exploitation may lead to premature localisation around suboptimal regions. From an inference perspective, the posterior distribution $p(\theta \mid y_{1:T}, \xi_{1:T})$ is typically multi-modal at early stages and becomes sharply concentrated as informative measurements accumulate. Thus, in this setting, amortised methods such as DAD aim to query maximally informative sensors and should generate sequences of designs that are effective for the prediction task.

Parameter	Value
Number of sources, K	2
$\dim_\xi$	2
$\dim_y$	1
T	30
Base signal, b	10^{-1}
Max. signal, m	10^{-4}
A	1
Signal noise, σ	0.5

Table 10: Location Finding Parameters

Downstream Objective. In this experimental setting, the downstream objective is to predict the source locations. Prediction quality is evaluated using two losses: the mean squared error (MSE) and a logarithmic variant, denoted Log-MSE. For problems with multiple sources, the source labels become exchangeable, meaning that permuting their ordering leaves both the likelihood and the

signal observed by the sensor unchanged. Hence, the downstream task is intrinsically permutation-invariant. What matters is the predicted set of source locations, not the arbitrary ordering assigned to its elements. Consequently, the standard MSE loss is not the most relevant evaluation criterion in this setting, since it imposes a fixed correspondence between predicted and true sources and may penalize predictions that are correct up to a permutation. Therefore, we use in this experiment the permutation invariant MSE over the losses, defined as

$$\ell_{MSE}(\theta, \hat{\theta}) = \min_{\sigma \in \mathcal{S}_K} \sum_{k=1}^K \|\hat{\theta}_{\sigma(k)} - \theta_k\|^2, \quad (55)$$

where $\mathcal{S}_K$ is the set of permutations of $\{1, \dots, K\}$ and $\hat{\theta}$ the point prediction of source locations. This optimization problem is significantly more complex than in the single-source case. The expectation involves a minimization over permutations inside the loss, leading to a highly non-linear and generally non-convex objective. This phenomenon poses significant challenges during training:

- **Label switching and multimodality.** The posterior distribution is invariant under permutations of the sources, which typically results in a multimodal posterior with $K!$ symmetric modes.
- **Failure of the posterior mean.** Averaging over these modes leads to a posterior mean that does not correspond to any meaningful configuration of sources, often placing predicted sources between true locations or collapsing them toward the center of mass.

As a result, under this permutation-invariant loss, the Bayes estimator is not the usual posterior mean on the labelled parameter space, but rather the posterior Fréchet mean induced by the optimal-assignment metric. Equivalently, the Bayes-optimal action is a fixed point where each predicted source is equal to the posterior mean of the true sources optimally matched to it; see Appendix F.3. This loss is adopted because it directly reflects the objective of the experiment, namely recovering the unordered set of source locations. For the Log-MSE loss, however, we avoid this additional complication by training the downstream network to solve a harder labelled prediction problem, with the loss defined as

$$\ell_{\text{Log-MSE}}(\mathcal{C}(\theta), \theta) = \|\mathcal{C}(\theta) - \hat{\theta}\|^2 + \log(\|\mathcal{C}(\theta) - \hat{\theta}\|^2 + \epsilon) \quad (56)$$

where $\epsilon = 10^{-6}$ is used for numerical stability, and $\mathcal{C}(\cdot)$ denotes a **canonicalisation operator** that enforces a deterministic ordering of the sources. Specifically, for a multi-source configuration $\theta = (\theta_1, \dots, \theta_K)$, we define

$$\mathcal{C}(\theta) = (\theta_{\pi(1)}, \dots, \theta_{\pi(K)}), \quad \pi = \underset{k}{\operatorname{argsort}} \|\theta_k\|_2,$$

so that sources are ordered by their Euclidean distance to the origin. This transformation maps each unordered set of sources to a canonical representative, thereby breaking the permutation symmetry and assigning a stable identity to each source. Consequently, the downstream network is trained to learn a well-defined mapping from trajectories $\{(\xi_i, y_i)\}_{i=1}^T$ to ordered source configurations.

Experimental Details. In this experiment, we consider three baselines methods:

1. **RANDOM**, where designs are naively sampled at random from the marginal prior distribution of a single source location.
2. **DAD**, the standard deep policy-based method introduced in [22]. It parameterises the adaptive design policy with a neural network and is trained using its native expected-information-gain objective. To ensure a fair comparison, we replace the architectural backbone of the original implementation with the same policy network used in our method.
3. **ALINE**, a recent policy-based method that achieves among the strongest reported performance in the literature. Its effectiveness is driven by a TNP-like policy architecture, a dense trajectory-level reward objective, and an internal posterior-approximation mechanism that amortises posterior inference. We retain the architecture proposed in the original paper, which is already close in spirit to the transformer-based policy network used in our method.

To compare methods on this task, each baseline is first trained using its native design objective, derived from an EIG-based criterion, thereby learning an acquisition policy independently of the

downstream predictor. Once this policy is fixed, a downstream network is trained on trajectories generated by the learned policy to predict the source locations from the resulting design–observation history. All methods are then evaluated through the predictive accuracy of this downstream network, ensuring that the comparison reflects performance on the final source-localization task rather than on the surrogate design objective alone.

The objective of this experiment is threefold. First, we evaluate our method from the perspective of downstream predictive performance. In particular, we assess whether the learned design policy produces informative trajectories that enable the downstream network to accurately solve the source-location point-estimation problem. Second, we evaluate the quality of the learned design policy using the usual lower and upper bounds on the EIG, estimated respectively by sPCE and sNMC. Third, we compare methods in terms of computational cost, measured by the wall-clock training time on a single GPU. We distinguish between the cost of learning the design policy itself and the total training cost, which, for the baselines, also includes the additional training of the downstream prediction network.

Architectures. We use the same policy-network architecture for ACTION-BED and DAD, detailed in Appendix E.1. At the start of each rollout, before any design–outcome observations are available, the initial history encoding is parameterised as a learnable vector. The only architectural hyperparameters varied across methods are the encoding dimensions, which are reported in Table 11.

Table 11: Policy architecture encoding dimensions.

Method	$d_{\text{enc},\xi}$	$d_{\text{enc},y}$	$d_{\text{enc},r}$	$d_{\text{enc},h}$
DAD	32	32	32	64
ACTION-BED	32	32	32	64

In this task, weight sharing between the design policy and the downstream network did not yield a clear performance improvement without further optimisation. We therefore use a simple fully connected MLP for the downstream network, which maps the design–outcome history $\{(\xi_i, y_i)\}_{i=1}^T$ to a point estimate of the source locations. The architecture is detailed in Table 12.

Table 12: Downstream network architecture.

Layer	Description	Dimension	Activation
Input	h_T	$T \times (\dim_\xi + \dim_y) = 90$	–
Hidden layer 1	Fully connected	512	GELU
Hidden layer 2	Fully connected	256	GELU
Hidden layer 3	Fully connected	128	GELU
Output	$\hat{\theta}$	4	–

Training procedure. During training, all policy networks are optimised from data generated online by rollouts of the corresponding design policy. At each iteration, latent parameters θ are sampled from the prior distribution $p(\theta)$, and the policy is rolled out autoregressively over the experimental horizon. The hyperparameters used for the policy networks are reported in Table 13. ALINE is implemented using its native architecture and trained with the hyperparameters reported in the original paper for the same source-location finding task. In particular, ALINE is trained for 100 000 iterations with a joint design policy and posterior approximation network, after an initial burning phase of 20 000 iterations during which only the posterior approximation network is optimised. DAD and Action-BED are implemented using the hyperparameters reported in the same table. For Action-BED, we additionally use a warm-up phase in which the downstream prediction network is trained in isolation to infer the source locations from randomly generated designs. After this warm-up, the design and downstream networks are optimised jointly using a shared optimizer. This pretraining stage provides a more informative learning signal to the design policy at the beginning of joint optimization, improves the coupling between the acquisition policy and the downstream predictor, and empirically leads to convergence toward better optima.

Evaluation procedure. After training, we evaluate the downstream action networks associated with each learned design policy, either jointly trained with the acquisition policy in Action-BED,

Table 13: **Hyperparameters for source location finding.** Training and optimization hyperparameters used for the policy networks and the downstream prediction networks. ExpLR denotes an exponential learning-rate scheduler. The RANDOM downstream network uses the same hyperparameters as DAD.

DAD		ALINE		Action-BED	
Field	Value	Field	Value	Field	Value
Policy training		Policy training		Joint training	
Batch size	1024	Batch size	200	Batch size	2000
Contrastive samples L	2000	Policy steps	100K	Joint steps	150K
Policy steps	150K	Burn-in steps	20K	Warm-up steps	50K
Policy seed	42	Burn-in training	Posterior only	Warm-up training	Downstream only
Policy optimization		Query pool, burn-in	30	Warm-up designs	Random
Optimizer	Adam	Query pool, train	200	Seed	42
Adam betas	(0.8, 0.998)	Query proposal	Uniform	Optimization	
Policy LR	10^{-3}	Reward discount	1	Optimizer	Adam
Scheduler	ExpLR	Policy weight α	1	Adam betas	(0.8, 0.998)
Decay γ	0.95	Policy seed	123	Policy LR	5×10^{-4}
Decay period	2000	Policy optimization		Scheduler	ExpLR
Weight decay	0	Optimizer	Adam	Decay γ	0.95
Downstream training		Adam betas	(0.9, 0.999)	Decay period	2000
Downstream steps	150K	Policy LR	10^{-3}	Weight decay	0
Downstream seed	42	Policy sched.	Cosine		
Downstream LR	5×10^{-4}	Grad. clipping	$\ \nabla\ _{\infty} \leq 1$		
Downstream sched.	ExpLR	Downstream training			
		Downstream steps	100K		
		Downstream LR	5×10^{-4}		
		Downstream sched.	ExpLR		
		Downstream decay γ	0.95		
		Decay period	2000		
		Downstream seed	42		

or trained independently after the design policy has been learned for other baselines. We evaluate downstream predictors using the two downstream losses described above. Evaluation rollouts are generated by first sampling latent source parameters $\theta \sim p(\theta)$ from the prior and then rolling out the corresponding adaptive design policy autoregressively over the full experimental horizon. The downstream network receives the resulting terminal history h_T and outputs a point prediction of the source configuration. All reported downstream losses are computed over 2048 independent evaluation rollouts, and uncertainty is reported as ± 1 standard error in Table 1.

EIG-bound evaluation. For each trained design policy, we additionally estimate lower and upper bounds on the EIG. This provides a standard, task-agnostic measure of design quality commonly used in the BED literature. It is not the downstream objective considered in our experiment, but DAD and ALINE are explicitly trained with information-theoretic objectives related to EIG maximisation. We recall the definition of these bounds:

$$\mathcal{L}_{\text{sPCE}} = \mathbb{E} \left[\log \frac{p(h_T | \theta_0; \pi_d^\phi)}{\frac{1}{L+1} \sum_{\ell=0}^L p(h_T | \theta_\ell; \pi_d^\phi)} \right], \quad (57)$$

$$\mathcal{U}_{\text{sNMC}} = \mathbb{E} \left[\log \frac{p(h_T | \theta_0; \pi_d^\phi)}{\frac{1}{L} \sum_{\ell=1}^L p(h_T | \tilde{\theta}_\ell; \pi_d^\phi)} \right], \quad (58)$$

with $\theta_0 \sim p(\theta)$, $\theta_{1:L} \sim p(\theta)$ are independent Monte Carlo samples used to approximate the marginal likelihood, and L being the number of contrastive samples drawn. For all methods, we use a large number of contrastive samples in both estimators so that the resulting bounds are sufficiently tight for comparison, see Table 14. Reported uncertainties correspond to ± 1 standard error over independent rollouts.

G.1.1 Qualitative Results

We complement the quantitative evaluation with qualitative trajectory plots of the learned design policies. We roll out each trained policy and visualize the sequence of selected designs over the experimental horizon. This comparison illustrates the characteristic behavior induced by each method: how quickly it explores the design space, whether it concentrates measurements around informative

Table 14: **Source location finding.** Evaluation protocol for design policies.

Method	Training objective	sPCE / sNMC		Downstream losses
		L	Batch size	Batch size
Action-BED (MSE)	Downstream loss	10^6	2000	2000
Action-BED (Log)	Downstream loss	10^8	200	2000
DAD	EIG-oriented objective	10^6	2000	2000
ALINE	EIG-oriented objective	5×10^6	500	2000
Random	None	10^5	2000	2000

Notes. L denotes the number of contrastive samples used to estimate the sequential PCE and NMC bounds. The corresponding batch size is the number of rollout histories used for these EIG-bound estimates. Downstream losses are evaluated on independent rollout batches.

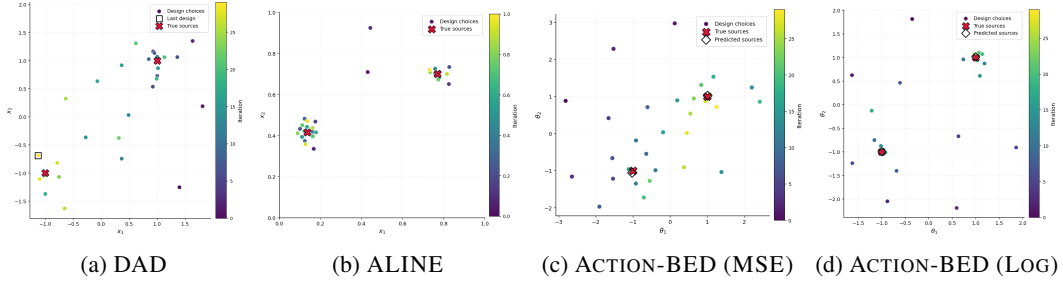


Figure 4: **Comparison of four policy rollouts.** Each panel shows one experimental design trajectory under the trained design policy.

regions, and how its sampling strategy adapts as observations are collected. These trajectories can provide an interpretable diagnostic of the experimental strategies learned by the different policies.

G.1.2 Amortising Downstream Bayes-action Selection

This section details the experimental protocol used to compare trained design policies on the downstream task, in terms of the quality of the Bayes-optimal action induced by the experimental trajectories they generate. This serves two purposes. First, it provides a direct way to compare design policies under the decision-theoretic objective of interest, namely the losses incurred by the action selected after observing the experimental history. Second, it illustrates a key practical advantage of ACTION-BED: since the design policy is trained jointly with an amortised downstream action policy, it directly learns an action network that predict a near-optimal decision, thereby amortising the otherwise costly posterior-inference step. Indeed, for design policies trained on the EIG, the downstream action-selection step is not directly available from training. Therefore, in order to evaluate such policies on downstream performance, one must proceed in two stages: first generate histories using the fixed trained design policy, and then infer or learn the corresponding downstream action from these histories. This is precisely the comparison performed here.

In this experiment, we evaluate the posterior Fréchet mean under both the MSE and Log-MSE losses. For the MSE objective, this estimator coincides with the Bayes action; see Appendix F. This provides a useful explicit posterior-based decision rule against which amortised downstream prediction can be compared. For each trained design policy, we compute the posterior Fréchet mean from posterior samples conditioned on the observed trajectory, and report its empirical downstream performance over the evaluation batch under both loss functions.

Inference protocol. To compute the explicit posterior Bayes action, we consider three posterior-mean estimators. The first one is based on importance sampling with the prior as proposal distribution. For each experimental history $h_T = (\xi_{1:T}, y_{1:T})$ generated by a fixed design policy, we sample particles $\theta^{(1)}, \dots, \theta^{(N)} \sim p(\theta)$ independently from the prior distribution. These particles are then reweighted according to the likelihood of the observed history,

$$w^{(i)}(h_T) \propto p(h_T | \theta^{(i)}), \quad \sum_{i=1}^N w^{(i)}(h_T) = 1.$$

This yields a weighted empirical approximation of the posterior distribution,

$$p(\theta \mid h_T) \approx \sum_{i=1}^N w^{(i)}(h_T) \delta_{\theta^{(i)}}.$$

However, because the latent sources are exchangeable, the direct posterior mean is not Bayes-optimal for the permutation-invariant MSE loss considered and may yield a degenerate estimate of the underlying sources by averaging across permutations. To account for this symmetry, we first optimally permuting the sources of each particle. Given a reference ordering $\bar{\theta}$, for instance the current empirical estimate or an initial point estimate, each particle $\theta^{(i)} = (\theta_1^{(i)}, \theta_2^{(i)})$ is aligned by solving

$$\sigma_i^* \in \arg \min_{\sigma \in \mathcal{S}_2} \sum_{k=1}^2 \left\| \theta_{\sigma(k)}^{(i)} - \bar{\theta}_k \right\|^2,$$

where $\mathcal{S}_K$ denotes the set of permutations of the latent sources. We then define the aligned particle

$$\tilde{\theta}^{(i)} = \left(\theta_{\sigma_i^*(1)}^{(i)}, \theta_{\sigma_i^*(2)}^{(i)} \right),$$

and estimate the MSE Bayes action by the weighted posterior Fréchet mean

$$\hat{a}_{\text{IS}}(h_T) = \sum_{i=1}^N w^{(i)}(h_T) \tilde{\theta}^{(i)}.$$

This aligned posterior mean is then used as the downstream prediction, and its empirical loss is evaluated on the held-out batch. The numerical parameters used for this importance-sampling estimator are reported in Table ??.

The second estimator is based on Hamiltonian Monte Carlo (HMC) [54]. HMC constructs a Markov chain targeting the posterior distribution by simulating Hamiltonian dynamics in an augmented position–momentum space. For each history h_T , we use HMC to draw approximate samples from the posterior distribution. In practice, the HMC chain is initialised from a coarse approximation of the optimal action obtained by a lighter version of the prior-proposal importance-sampling procedure described above. This provides a reasonable starting point near a high-posterior-probability region and stabilises the subsequent sampling stage. After burn-in and thinning according to the parameters reported in Table ??, we obtain posterior samples for each observed trajectory. The HMC-based Bayes action is then computed using the same post-processing procedure as for importance sampling. The empirical posterior mean is then computed over the aligned samples and used as the downstream MSE Bayes-action estimate. We finally evaluate the empirical downstream MSE incurred by this prediction over the evaluation batch.

The third estimator is based on a Laplace approximation of the posterior distribution. For each history h_T , we first compute a maximum-a-posteriori estimate of the latent parameters by optimising the log posterior,

$$\hat{\theta}_{\text{MAP}}(h_T) \in \arg \max_{\theta} \{ \log p(h_T \mid \theta) + \log p(\theta) \}.$$

In practice, the optimisation is initialised from a coarse particle-based approximation in order to start near a high-posterior-probability region, and multiple restarts are used to improve robustness. The posterior is then locally approximated by a Gaussian distribution centred at the MAP, with precision given by the negative Hessian of the log posterior at this point,

$$p(\theta \mid h_T) \approx \mathcal{N} \left(\hat{\theta}_{\text{MAP}}(h_T), (H(h_T) + \lambda_{\text{ridge}} I)^{-1} \right),$$

where $H(h_T)$ denotes the local posterior precision and λ_{ridge} is a small ridge regularisation parameter added for numerical stability. We then draw samples from this Gaussian approximation using a Cholesky decomposition whenever the regularised precision matrix is positive definite, as described in Table ??. The Laplace-based Bayes action is finally computed using the same post-processing procedure as for importance sampling and HMC: the Gaussian samples are aligned to account for the exchangeability of the latent sources, and the empirical posterior mean over the aligned samples is used as the downstream MSE Bayes-action estimate. We finally evaluate the empirical downstream MSE incurred by this prediction over the evaluation batch.

(a) Hamiltonian Monte Carlo		(b) Laplace approximation	
Parameter	Value	Parameter	Value
Evaluation batch size	2000	Evaluation batch size	2000
HMC samples	5×10^4	MAP optimisation steps	1000
Burn-in steps	10^4	Laplace samples	5×10^4
Step size ϵ^*	$[10^{-3}, 10^{-2}]$	Ridge regularisation λ_{ridge}	10^{-3}
Leapfrog steps	10	Factorisation	Cholesky Decomposition
Thinning interval	1		
PM init. particles N_{PM}	10^5		

(c) Prior-proposal IS	
Parameter	Value
Particles N	5×10^6
Evaluation batch size	2000

Table 15: **Hyperparameters for posterior Bayes-action estimation.** We report the parameters used for the two Bayes-action estimators: prior-proposal IS and HMC. The HMC step size ϵ is adapted separately for each method so as to match a target acceptance rate, while remaining constrained to the interval reported in the table.

Indicative inference cost. Since posterior-action estimators require test-time inference for each observed trajectory, we additionally report, in Table 17, the wall-clock time required to infer actions and evaluate the downstream loss on a batch of 2000 trajectories. These timings were obtained on the evaluation machine used for this posterior-inference study, which differs from the hardware used for training. They should therefore be interpreted as indicative deployment costs rather than hardware-normalised runtime benchmarks. Nevertheless, they highlight the qualitative distinction between amortised action prediction, which only requires a forward pass through the downstream network, and posterior-inference baselines, which require iterative or sampling-based computation at test time.

Discussion. The results reported in Table 16 highlight an important distinction between the two downstream losses considered in this work. Although both losses evaluate the quality of a final action inferred from the experimental history, they encode different downstream objectives. Consequently, the corresponding Bayes actions are not the same, and neither are the design policies learned by ACTION-BED. This distinction is crucial when interpreting the posterior-action evaluations of fixed design policies. The experimental trajectories produced by the policy are shaped by the final decision criterion they are meant to serve. ACTION-BED trained with the Log-MSE loss, as shown in Table 1, achieves strong Log-MSE performance, and its learned designs often concentrate tightly around the latent sources. One might therefore expect that, under these highly localised designs, the MSE Bayes action obtained by posterior inference would also be especially accurate, perhaps even outperforming the policies trained with other objectives. However, this is not what we observe in practice.

As illustrated in Figure 5, the distribution of rollout-wise MSE losses differs substantially between ACTION-BED (MSE) and ACTION-BED (LOG-MSE). Under the same evaluation conditions, ACTION-BED (LOG-MSE) achieves markedly lower losses for a large fraction of rollouts: in particular, its median MSE is substantially smaller, indicating that most of its downstream predictions are highly accurate. However, this improvement over the bulk of the distribution comes at the cost of a much heavier upper tail. For the largest quantiles, the MSE losses of ACTION-BED (LOG-MSE) deteriorate sharply and become much larger than those obtained by ACTION-BED (MSE). Thus, although the Log-MSE-trained policy produces very accurate predictions on the majority of trajectories, it also induces rare rollouts on which the downstream action can fail severely. These high-loss trajectories dominate the average MSE and explain why the overall MSE performance of the Log-MSE-trained policy is worse despite its stronger median behaviour.

ACTION-BED (LOG-MSE) learns to generate efficient experimental trajectories and an associated amortised action rule that are well adapted to the geometry of the Log-MSE objective: most predictions are brought extremely close to the true sources, but robustness to rare high-error trajectories is not prioritised in the same way as under the MSE objective. By contrast, the MSE-trained policy learns designs that may be less sharply accurate on the typical rollout but better control the upper

Table 16: **Deployment-time Bayes-action estimation under fixed acquisition policies.** For each pretrained design policy, we compare amortised action networks with posterior-based Bayes-action estimators. This evaluates how well each policy supports downstream decision-making after observing an experimental history. Uncertainties are reported as one standard error over 2048 rollouts.

Acquisition	Decision Rule	Performance (± 1 s.e.)	
		MSE (10^{-2}) $\downarrow$	Log-MSE $\downarrow$
ACTION-BED (MSE)	Native network (MSE)	0.55 ± 0.06	-6.83 ± 0.03
	Retrained network (MSE)	0.86 ± 0.09	-6.93 ± 0.01
	Bayes-act. (IS)	0.44 ± 0.03	-7.25 ± 0.03
	Bayes-act. (HMC)	0.61 ± 0.04	-7.11 ± 0.02
	Bayes-act. (Laplace)	0.46 ± 0.03	-7.33 ± 0.04
ACTION-BED (Log)	Native network (Log)	2.8 ± 0.4	-10.10 ± 0.03
	Retrained network (Log)	1.7 ± 0.2	-9.97 ± 0.03
	Bayes-act. (IS)	1.3 ± 0.6	-9.37 ± 0.04
	Bayes-act. (HMC)	2.00 ± 0.43	-10.12 ± 0.04
	Bayes-act. (Laplace)	2.31 ± 0.71	-10.32 ± 0.05
DAD	Retrained network	5.6 ± 0.6	-6.58 ± 0.05
	Bayes-act. (IS)	3.9 ± 0.2	-8.06 ± 0.05
	Bayes-act. (HMC)	3.6 ± 0.5	-8.11 ± 0.05
	Bayes-act. (Laplace)	3.5 ± 0.8	-8.05 ± 0.06
ALINE	Retrained network	0.21 ± 0.03	-8.11 ± 0.07
	Bayes-act. (IS)	0.16 ± 0.05	-9.22 ± 0.03
	Bayes-act. (HMC)	0.35 ± 0.21	-9.17 ± 0.03
	Bayes-act. (Laplace)	0.13 ± 0.04	-9.74 ± 0.07
Random	Retrained network	22.4 ± 0.8	-3.16 ± 0.04
	Bayes-act. (IS)	6.1 ± 0.7	-4.36 ± 0.05
	Bayes-act. (HMC)	15.7 ± 2.5	-3.45 ± 0.08
	Bayes-act. (Laplace)	32.4 ± 3.1	-3.28 ± 0.09

Notes. Retrained networks are trained post-hoc on histories generated by the fixed design policy, whereas native networks for ACTION-BED are jointly trained on the downstream training loss. Bayes-action estimators use the fixed acquisition policy and compute the deployment-time action from the posterior induced by the observed history.

Table 17: **Indicative test-time cost of posterior-action inference.** We report the approximate wall-clock time required to infer actions and evaluate the downstream loss on a batch of 2000 trajectories.

Action Estimator	Approx. time (seconds) $\downarrow$
Amortised downstream network	≈ 6
Importance-sampling posterior mean	≈ 7600
Laplace posterior approximation	≈ 250
HMC posterior approximation	≈ 3100

Notes. Timings include action inference and loss evaluation. They are measured on a separate evaluation machine and are intended only to indicate the approximate order of magnitude of the deployment cost.

tail of the squared error distribution. This also explains why ACTION-BED trained with Log-MSE remains inferior under the MSE criterion, even when replacing its amortised action network by posterior-inference-based Bayes actions.

G.1.3 Additional Results

Training time decomposition. All training-time comparisons are performed on the same GPU, so that reported differences reflect the computational cost of the methods rather than hardware variability. In addition to the main performance metrics, we report in Table 18 a decomposition of the training time into policy and downstream action-network training. This decomposition shows that DAD has

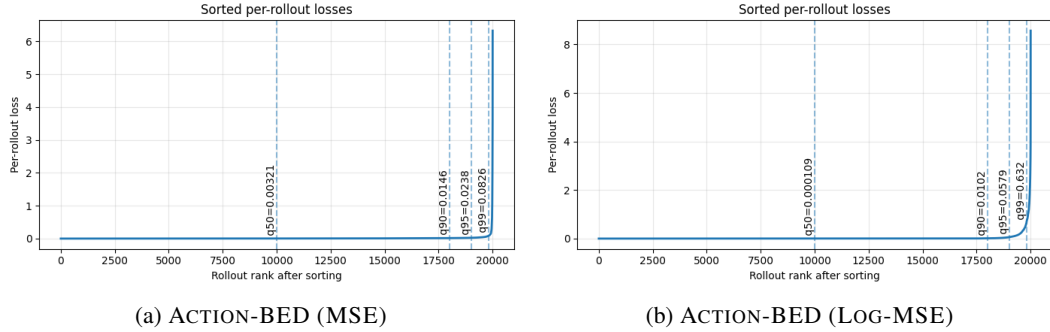


Figure 5: **Distribution of rollout-wise MSE losses for loss-specialised ACTION-BED policies.** We report per-rollout downstream MSE losses sorted from smallest to largest for ACTION-BED trained with the MSE objective and with the Log-MSE objective.

a slightly larger total training time than Action-BED, although its policy network alone is faster to train than the jointly trained Action-BED model. This is likely due to the batch sizes used in this experiment and to the relatively large neural architectures, for which backpropagation represents a substantial fraction of the total cost. Consequently, the computational advantage of the singly intractable objective is partly diluted by the cost of the network updates. ALINE is markedly more expensive: policy training alone already exceeds the full joint-training time of Action-BED, and its downstream stage remains computationally demanding because adaptive rollouts are costly to generate. Due to its simplicity, the random-design baseline is comparatively inexpensive to train.

Evolution of downstream performance. Figure 6 reports an additional time-resolved comparison of downstream performance during policy training. For each method, we periodically freeze the design-policy weights and evaluate the downstream losses using the same evaluation protocol as above. For DAD and ALINE, this requires training a downstream action network from scratch at each frozen policy checkpoint, using the same architecture and training hyperparameters as reported in Tables 12 and 13. For Action-BED, we instead evaluate the downstream action network associated with the joint model at the corresponding checkpoint. Importantly, the horizontal axis only accounts for the elapsed policy-training time, or joint-training time for Action-BED, including the potential warm-up time. Thus, it excludes the additional cost of training independent downstream action networks for the baselines. Checkpoints are shown up to convergence. For the joint Action-BED models, both downstream losses are evaluated at every checkpoint, even when the downstream predictor was not explicitly optimised for the loss under consideration.

This experiment illustrates how quickly each design policy produces histories that are useful for downstream prediction. Under the permutation-invariant MSE, Action-BED rapidly reaches a regime in which its action network performs well, achieving the best performance during approximately the first six hours of training. At later checkpoints, however, downstream networks trained on ALINE policies achieve stronger performance. Under the log-MSE loss, Action-BED trained with the log-MSE objective yields substantial gains very early in training, outperforming the other methods by a large margin.

Effect of Joint Training on Policy Co-Adaptation. We further examine whether joint training induces a useful co-adaptation between the design policy network and the downstream action policy network. To isolate this effect, we take a fully trained ACTION-BED design policy, keep it fixed, and retrain a new downstream action network under the same optimisation budget and hyperparameters as in the original joint training procedure; see Tables 12 and 13. We perform this retraining separately with the MSE and Log-MSE objectives.

The relevant comparison is the same-loss setting: for each ACTION-BED model, we compare the native jointly trained action network to a post-hoc action network retrained with the same loss used to learn the design policy. As shown in Table 19, this retrained network performs slightly worse than the native joint network, despite being trained on histories generated by an already fully optimised design policy. This suggests that the gain of joint training is due to the optimisation path through which the design and action networks become mutually adapted. In particular, the design policy learns to generate histories that are informative in a form the action network can exploit, while the

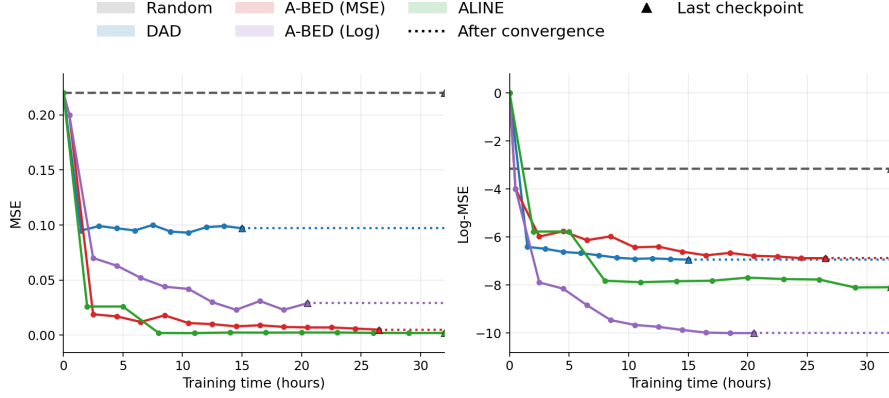


Figure 6: Comparison of downstream losses as a function of design policy training time, until convergence.

Table 18: Training Time Decomposition.

Method	Design	Downstream	Total
ACTION-BED (MSE)	–	–	1171
ACTION-BED (Log)	–	–	1184
DAD (MSE)	879	335	1214
DAD (Log)	879	337	1216
ALINE (MSE)	1780	1182	2962
ALINE (Log)	1780	1190	2970
Random	0	31	31

Notes. Wall-clock GPU times are reported in minutes. For ACTION-BED, design and downstream action networks are trained jointly, so only total training time is reported. Parentheses indicate the loss used to train the downstream action network.

action network simultaneously learns to decode both the observed outcomes and the information implicit in the adaptive sequence of designs.

Table 19: Native versus retrained downstream networks for A-BED. Uncertainties are reported as one standard error.

Acquisition	Action Network	Performance (± 1 s.e.)	
		MSE (10^{-2}) $\downarrow$	Log-MSE $\downarrow$
ACTION-BED (MSE)	Native network	0.55 ± 0.06	-6.83 ± 0.03
	Retrained network	0.86 ± 0.09	-6.93 ± 0.01
ACTION-BED (Log)	Native network	2.8 ± 0.4	-10.10 ± 0.03
	Retrained network	1.7 ± 0.2	-9.97 ± 0.03

Notes. Native networks are obtained directly from joint Action-BED training. Retrained networks are trained post-hoc after freezing the learned acquisition policy, using the same downstream loss. MSE is reported in units of 10^{-2} .

Cross-Evaluation of Design and Action Policies. We further study the coupling between acquisition and decision-making by cross-evaluating all trained downstream action networks across all design policies in the benchmark. For each trained design policy, we generate histories under that policy and evaluate the predictions produced by action networks trained with every other method with the MSE loss. For ACTION-BED, the action network is the one learned jointly with the design policy; for the other baselines, it is the network trained subsequently on trajectories from the fixed design policy. Table 20 reports normalised performance ratios, measuring, for each fixed design policy, the

degradation incurred when replacing its associated action network by an action network trained on another history distribution.

This experiment tests whether action policies learn transferable Bayes-optimal action, or instead become specialised to the distribution of histories induced by their own design policy. Design policies that are more accurate and concentrate their measurements tightly around the latent sources induce history distributions on which action networks trained under other acquisition strategies perform substantially worse. In this regime, the downstream predictor must exploit fine-grained, policy-specific patterns in the adaptive design sequence, rather than only aggregate generic information from the observations. Conversely, the action networks paired with these more focused design policies achieve strong performance on their own histories but transfer poorly to histories generated by other policies. This suggests that high-performing methods do not merely collect more information; they also shape the information in a form that is specifically matched to their associated downstream decision rule.

Table 20: **Normalized cross-policy coupling between acquisition and action networks.** Rows correspond to the acquisition policy used at test time, while columns correspond to the downstream action network used for decision-making.

Test Acquisition	Downstream Action Network				
	A-BED (MSE)	A-BED (Log)	DAD	ALINE	Random
ACTION-BED (MSE)	1.00	304.55	114.73	175.64	13.64
ACTION-BED (Log)	17.11	1.00	16.07	34.75	9.21
DAD	19.48	1079.98	1.00	16.89	6.14
ALINE	32.86	18.10	20.03	1.00	131.91
Random	11.83	1135.97	4.16	10.82	1.00

Notes. Each entry reports a normalized degradation factor. For a test acquisition policy π_d^i and an action network π_a^j , we define $\tilde{R}_{ij} = R(\pi_d^i, \pi_a^j) / \min_k R(\pi_d^i, \pi_a^k)$.

Impact of Warmstarting in Joint Training. We assess whether warmstarting the downstream action network improves joint ACTION-BED training. Table 21 shows that, for both the MSE and Log-MSE objectives, warmstarting leads to better final performance. This suggests that the benefit is not limited to a faster initial optimisation of the action network. Rather, by providing a more accurate downstream predictor early in training, warmstarting yields a more informative learning signal for the design policy. This improved signal changes the subsequent joint optimisation dynamics and guides the coupled design–action system toward a better final solution.

Table 21: **Impact of warmstarting in joint Action-BED training.** Uncertainties are reported as one standard error.

Acquisition	Warmstart	Performance (± 1 s.e.)				Compute Time
		MSE (10^{-2}) $\downarrow$	Log-MSE $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$	GPU (min) $\downarrow$
ACTION-BED (MSE)	Yes	0.55 $\pm$ 0.06	−6.83 $\pm$ 0.03	11.63 $\pm$ 0.04	12.84 $\pm$ 0.08	1171
	No	0.87 $\pm$ 0.1	−6.82 $\pm$ 0.01	11.62 $\pm$ 0.08	12.67 $\pm$ 0.15	1159
ACTION-BED (Log)	Yes	2.8 $\pm$ 0.4	−10.10 $\pm$ 0.03	17.14 $\pm$ 0.11	21.07 $\pm$ 0.45	1184
	No	3.0 $\pm$ 0.3	−9.33 $\pm$ 0.03	16.48 $\pm$ 0.18	18.88 $\pm$ 0.49	1172

G.2 Dynamical Systems

We consider sequential design problems in which the underlying simulator is a dynamical system. Unlike exchangeable observation models, each outcome depends on the current system state, so the order in which observations are collected matters. This setting is adapted from [38] and covers both the simple- and double-pendulum experiments considered in this work.

More precisely, we view these experiments as an abstraction of dynamical systems governed by stochastic differential equations, detailed below, whose observation model is Markovian:

$$p(y_{t+1} \mid y_t, \xi_{t+1}, \theta),$$

so that the next observation is generated from the previous state or observation y_t , the current design ξ_{t+1} , and the physical parameters θ . Hence, the likelihood is sequential and cannot be factorized as with conditionally exchangeable observations. Under these assumptions, the joint distribution of a trajectory can be written as

$$\begin{aligned} p(h_T \mid \theta; \pi_d^\phi) &= \prod_{t=1}^T p(y_t \mid h_{t-1}, \theta) \\ &= \prod_{t=1}^T p(y_t \mid y_{t-1}, \xi_t, \theta). \end{aligned}$$

Architectures. For both pendulum tasks, we use the same TNP-like policy architecture for ACTION-BED and DAD, as described in Appendix E.1 and with the same encoding dimensions as in Table 11. Since the likelihood is non-exchangeable, we add a time embedding to each history element r_t before aggregation. At the beginning of a rollout, when no design–outcome pair has yet been observed, the initial history representation is parameterized as a learnable vector.

For both ACTION-BED and DAD, the emitter output is additionally mapped through an affine–tanh transformation,

$$\xi_t = a \tanh(\tilde{\xi}_t) + b,$$

which ensures that the selected designs lie in the admissible control domain. The same architectural template is used for the stochastic and double pendulum experiments; only the input and output dimensions can vary, depending on the dimension of the designs and observations.

ALINE is permutation-invariant by construction and is therefore not directly tailored to non-exchangeable likelihoods. To make the sequential structure explicit, we augment each context element with a time encoding and include the most recent observation in the context before encoding. This provides the model with the information needed to condition each candidate design on the current dynamical state. RANDOM samples designs uniformly at random over the valid constrained space.

In all cases, the downstream predictor is a fully connected MLP mapping the complete design–outcome history to a point estimate of θ in the relevant space (see Table 22). No parameter sharing is used between the design policy and the downstream network.

Table 22: Downstream network architecture.

Layer	Description	Dimension	Activation
Input	h_T	$T \times (\dim_\xi + \dim_y)$	–
Hidden layer 1	Fully connected	512	GELU
Hidden layer 2	Fully connected	256	GELU
Hidden layer 3	Fully connected	128	GELU
Output	$\hat{\theta}$	d_θ	–

Training procedure. For the dynamical-system experiments, we follow a training protocol closely mirroring that of the source location finding task. All policies are trained from trajectories generated by the corresponding simulator. At each iteration, latent parameters θ are sampled from the prior, the design policy is rolled out autoregressively over the experimental horizon, and each method is trained on its own objective. We compare several downstream objectives in order to assess the accuracy and flexibility of the learned predictive action network. We consider the standard mean-squared error on θ ,

$$\mathcal{L}_{\text{MSE}}(\hat{\theta}, \theta) = \|\hat{\theta} - \theta\|_2^2,$$

the logarithmic MSE,

$$\mathcal{L}_{\log\text{-MSE}}(\hat{\theta}, \theta) = \log\left(\|\hat{\theta} - \theta\|_2^2 + \varepsilon\right),$$

and a weighted MSE,

$$\mathcal{L}_{\text{wMSE}}(\hat{\theta}, \theta) = \sum_{j=1}^{d_\theta} w_j (\hat{\theta}_j - \theta_j)^2.$$

For the stochastic pendulum, we use weights

$$w = (0.1, 1.0, 2.0),$$

whereas for the double pendulum we use

$$w = (1.0, 1.0, 2.0, 3.0).$$

The weighted objective allows different components of θ to be emphasized, illustrating that the downstream target can be adapted to the scientific or operational quantity of interest.

Evaluation procedure. In addition to downstream prediction performance, we evaluate each trained policy using sequential PCE and sequential NMC bounds on the EIG, which provide a common information-theoretic measure of design quality across methods. For both dynamical systems and for all downstream losses, prediction metrics are estimated from 2048 rollout batches and reported with ± 1 standard error. For the single-pendulum experiment, the sequential PCE and NMC estimates are computed with $L = 50\text{K}$ contrastive samples and a batch size of 2000. For the double-pendulum experiment, we use a larger contrastive budget, with $L = 200\text{K}$ and a batch size of 1000. We also report the total GPU compute time required for training.

G.2.1 Stochastic Pendulum

The first dynamical system considered is a stochastic single pendulum. The simulator describes the trajectory of a pendulum whose state evolves over time according to noisy dynamics. At each step, the design variable corresponds to an external control applied to the system, allowing the policy to influence the subsequent motion of the pendulum and thereby collect informative observations about the latent physical parameters.

Task. We consider the problem of actively identifying the physical parameters of a stochastic controlled pendulum. The state is $x_t = (q_t, \dot{q}_t)$, where q_t denotes the angle from the vertical and $\dot{q}_t$ the angular velocity. The parameters of interest are the mass and length of the pendulum, (m, l) , while the gravitational acceleration and damping coefficient are fixed to $g = 9.81$, $d = 0.1$. The initial state is fixed as $x_0 = (0, 0)^\top$. At each step of the discretized dynamics, with time step $\Delta t = 0.05$, the policy selects a bounded control input $\xi_t \in [-1, 1]$, corresponding to a torque applied to the pendulum. The observation is the state of the system at the next discretization step. The objective is to choose controls that generate trajectories informative about the latent physical parameters.

Stochastic Dynamics. In continuous time, the controlled dynamics follow the stochastic differential equation

$$dx_t = h(x_t, \xi_t)^\top \theta dt + L d\beta_t,$$

where β_t is a Brownian motion,

$$h(x_t, \xi_t) = (-\sin q_t, -\dot{q}_t, \xi_t), \quad L = (0, 0.1)^\top.$$

Equivalently,

$$\begin{aligned} dq_t &= \dot{q}_t dt, \\ d\dot{q}_t &= h(x_t, \xi_t)^\top \theta dt + 0.1 d\beta_t. \end{aligned}$$

The observations used in the experiment are the discretized states

$$y_t = x_t = (q_t, \dot{q}_t),$$

recorded every $\Delta t = 0.05$. The resulting observation process is sequential: each new observation depends on the previous state, the applied control, and the latent parameter θ .

Following the conditionally linear formulation, the unknown parameter is

$$\theta = \left(\frac{3g}{2l}, \frac{3d}{ml^2}, \frac{3}{ml^2} \right)^\top,$$

which is bijectively related to the physical parameters (m, l) . We place the Gaussian prior

$$p(\theta) = \mathcal{N} \left(\begin{bmatrix} 14.7 \\ 0 \\ 3.0 \end{bmatrix}, \begin{bmatrix} 0.1 & 0 & 0 \\ 0 & 0.01 & 0 \\ 0 & 0 & 0.1 \end{bmatrix} \right).$$

The design variable ξ_t represents the **external torque** applied to the pendulum.

Table 23: **Hyperparameters for stochastic single pendulum.** Training and optimization hyperparameters used for the policy networks and the downstream prediction networks. ExpLR denotes an exponential learning-rate scheduler. The RANDOM downstream network uses the same hyperparameters as DAD.

DAD		ALINE		Action-BED	
Field	Value	Field	Value	Field	Value
Policy training		Policy training		Joint training	
Batch size	512	Batch size	200	Batch size	512
Contrastive samples L	2000	Policy steps	80K	Joint steps	50K
Policy steps	50K	Burn-in steps	20K	Warm-up steps	10K
Policy seed	42	Burn-in training	Posterior only	Warm-up training	Downstream only
Policy optimization		Query pool, burn-in	50	Warm-up designs	Random
Optimizer	Adam	Query pool, train	200	Seed	42
Adam betas	(0.8, 0.998)	Query proposal	Uniform	Optimization	
Policy LR	10^{-4}	Reward discount	1	Optimizer	Adam
Scheduler	ExpLR	Policy weight α	1	Adam betas	(0.8, 0.998)
Decay γ	0.96	Policy seed	123	Policy LR	10^{-4}
Decay period	400	Policy optimization		Scheduler	ExpLR
Weight decay	0	Optimizer	Adam	Decay γ	0.96
Downstream training		Adam betas	(0.9, 0.999)	Decay period	400
Downstream steps	50K	Policy LR	10^{-3}	Weight decay	0
Downstream seed	42	Policy sched.	Cosine		
Downstream LR	10^{-4}	Grad. clipping	$\ \nabla\ _\infty \leq 1$		
Downstream sched.	ExpLR	Downstream training			
		Downstream steps	50K		
		Downstream LR	10^{-3}		
		Downstream sched.	ExpLR		
		Downstream decay γ	0.95		
		Decay period	1000		
		Downstream seed	42		

Hyperparameters. The main training hyperparameters are reported in Table 23. ACTION-BED and DAD are trained for 50K gradient steps. For ACTION-BED, we additionally use a 10K-step warm-up phase in which the downstream network is trained on trajectories generated from random designs. ALINE is trained with a total budget of 100K steps, including a 20K-step burning phase during which only the posterior inference network is optimised.

G.2.2 Stochastic Double Pendulum

We next consider a controlled stochastic double-pendulum system. Compared with the single-pendulum experiment, this benchmark involves higher-dimensional, nonlinear, and coupled dynamics, making the generated trajectories substantially more sensitive to both the applied controls and the latent physical parameters. The state is

$$x_t = \begin{bmatrix} q_t \\ \dot{q}_t \end{bmatrix} = (q_{1,t}, q_{2,t}, \dot{q}_{1,t}, \dot{q}_{2,t})^\top \in \mathbb{R}^4,$$

where q_1, q_2 are the joint angles and $\dot{q}_1, \dot{q}_2$ their angular velocities. The unknown parameter is

$$\theta = (m_1, m_2, l_1, l_2)^\top,$$

corresponding to the masses and lengths of the two links. As in the single-pendulum setting, observations are the $T = 50$ discretized states of the system, collected along a trajectory. The design variable is a pair of bounded torques

$$\xi_t = (\xi_{1,t}, \xi_{2,t})^\top, \quad \xi_{1,t} \in [-4, 4], \quad \xi_{2,t} \in [-2, 2],$$

applied to the two joints. The downstream task is to infer θ from the resulting controlled trajectory.

Stochastic Dynamics. The double-pendulum dynamics follow the standard manipulator form

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + \tau_g(q) = \xi,$$

where $M(q)$ is the inertia matrix, $C(q, \dot{q})$ contains the Coriolis and centrifugal terms, and $\tau_g(q)$ is the gravitational torque. We use $g = 9.81$ and add process noise to the acceleration dynamics. The resulting stochastic differential equation is

$$dq_t = \dot{q}_t dt, \quad d\dot{q}_t = M(q_t)^{-1}(\tau_g(q_t) + \xi_t - C(q_t, \dot{q}_t)\dot{q}_t) dt + L d\beta_t,$$

Table 24: **Hyperparameters for stochastic double pendulum.** ExpLR denotes an exponential learning-rate scheduler. The RANDOM downstream network uses the same hyperparameters as DAD.

DAD		ALINE		Action-BED	
Field	Value	Field	Value	Field	Value
Policy training		Policy training		Joint training	
Batch size	512	Batch size	200	Batch size	512
Contrastive samples L	2000	Policy steps	80K	Joint steps	100K
Policy steps	50K	Burn-in steps	20K	Warm-up steps	20K
Policy seed	42	Burn-in training	Posterior only	Warm-up training	Downstream only
Policy optimization		Query pool, burn-in	50	Warm-up designs	Random
Optimizer	Adam	Query pool, train	150	Seed	42
Adam betas	(0.8, 0.998)	Query proposal	Uniform	Optimization	
Policy LR	10^{-4}	Reward discount	1	Optimizer	Adam
Scheduler	ExpLR	Policy weight α	1	Adam betas	(0.8, 0.998)
Decay γ	0.96	Policy seed	123	Policy LR	5×10^{-4}
Decay period	400	Policy optimization		Scheduler	ExpLR
Weight decay	0	Optimizer	Adam	Decay γ	0.96
Downstream training		Adam betas	(0.9, 0.999)	Decay period	400
Downstream steps	50K	Policy LR	10^{-3}	Weight decay	0
Downstream seed	42	Policy sched.	Cosine		
Downstream LR	10^{-4}	Grad. clipping	$\ \nabla\ _{\infty} \leq 1$		
Downstream sched.	ExpLR	Downstream training			
		Downstream steps	100K		
		Downstream LR	10^{-3}		
		Downstream sched.	ExpLR		
		Downstream decay γ	0.95		
		Decay period	1000		
		Downstream seed	42		

where $\beta_t = (\beta_{1,t}, \beta_{2,t})^\top$ is a two-dimensional Brownian motion with independent components and

$$L = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}.$$

The SDE is discretized with the same Euler–Maruyama scheme as in the single-pendulum experiment, with $\Delta t = 0.05$. The observation process is therefore Markovian and non-exchangeable: each new state depends on the previous state, the applied joint torques, and the latent physical parameters.

We place a log-normal prior on the physical parameters,

$$p(\theta) = \text{LogNormal} \left(\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, 0.01 I_4 \right),$$

so that masses and lengths remain positive. This task therefore tests whether adaptive design policies can select torque sequences that generate trajectories informative about coupled inertial, gravitational, and velocity-dependent effects.

Hyperparameters. The main training hyperparameters are reported in Table 24. For this task, ACTION-BED is trained for 100K gradient steps, preceded by a 20K-step warm-up phase in which only the downstream action network is optimised. ALINE is trained under the same total budget of 100K steps, including a 20K-step burn-in phase for the posterior inference network. Finally, DAD and its associated downstream networks are trained for 50K steps, as no further improvement was observed beyond this point. In all cases, downstream action networks are trained until their predictive performance saturates.

G.2.3 Additional Results

We report additional compute-time results for both dynamical-system tasks. Tables 25 and 26 decompose the total wall-clock GPU time into design-policy training and downstream-network training whenever these stages are trained separately.

Table 25: **Single Pendulum.** Training time decomposition.

Method	Design	Downstream	Total
ACTION-BED (MSE)	–	–	240
ACTION-BED (Log)	–	–	287
ACTION-BED (W-MSE)	–	–	290
DAD (MSE)	463	207	670
DAD (Log)	463	246	709
DAD (W-MSE)	463	208	671
ALINE (MSE)	1520	600	2120
ALINE (Log)	1520	621	2141
ALINE (W-MSE)	1520	602	2122
RANDOM	0	19	19

Table 26: **Double Pendulum.** Training time decomposition.

Method	Design	Downstream	Total
ACTION-BED (MSE)	–	–	501
ACTION-BED (Log)	–	–	504
ACTION-BED (W-MSE)	–	–	504
DAD (MSE)	862	72	934
DAD (Log)	862	78	940
DAD (W-MSE)	862	71	933
ALINE (MSE)	1938	1075	3013
ALINE (Log)	1938	1124	3062
ALINE (W-MSE)	1938	1069	3007
RANDOM	0	58	58

Notes. Wall-clock GPU times are reported in minutes. For ACTION-BED, the design and downstream action networks are trained jointly, so only total training time is reported. Parentheses indicate the loss used to train the downstream action network.

G.3 Masked MNIST Classification Experiment

We consider a sequential masked-classification task inspired by Iollo et al. [37] and based on the MNIST dataset [46]. The latent variable is an image $\theta \in \mathbb{R}^{28 \times 28}$, and the target of interest is its class label $z \in \{0, \dots, 9\}$. The goal is to identify the digit class from a small number of partial and noisy observations of the image.

At each step, the design $\xi \in [1, 28]^2$ specifies a continuous spatial location in pixel coordinates, interpreted as the top-left corner of a local patch. The observation is a noisy 5×5 patch extracted around this location:

$$y = A_\xi \theta + \eta, \quad \eta \sim \mathcal{N}(0, \sigma^2 I_d),$$

where $d = 5^2$, $A_\xi \theta$ denotes the extracted patch, and zero padding is used whenever the patch extends outside the image domain. This illustrates that the observations are reparametrizable, which allows gradients to be propagated through the observation process. Conditionally on the image, the observation model is therefore explicit:

$$p(y \mid \theta, \xi) = \mathcal{N}(A_\xi \theta, \sigma^2 I_d).$$

A direct implementation of A_ξ as a discrete masking operator would be non-differentiable with respect to ξ : small changes in the queried location would not affect the selected pixels until crossing an integer pixel boundary. To obtain a differentiable simulator, we instead use a continuous counterpart of the patch-extraction operator. In practice, the image is queried by grid sampling at the 5×5 patch coordinates induced by continuous ξ . When a sampling coordinate is not integer-valued, its intensity is obtained by bilinear interpolation from the four neighboring pixels. Thus, moving the design location smoothly changes the observed patch, making the simulator differentiable with respect to ξ while retaining the interpretation of localized pixel measurements.

Over a horizon $T = 5$, the policy adaptively selects patch locations $\xi_{1:T}$ based on the previously observed patches, with the objective of producing a final history informative for classification. This means that the task has a hierarchical structure. Observations are generated from the latent image θ , but the target of interest is the discrete label z . Consequently, the relevant class-conditional observation model is obtained by marginalizing over images within each class,

$$p(y | z, \xi) = \int_{\Theta} p(y | \theta, \xi) p(\theta | z) d\theta.$$

Although $p(y | \theta, \xi)$ is available in closed form, this class-conditional likelihood is not. In practice, the joint prior over images and labels is only available through the empirical dataset,

$$p(\theta, z) = \frac{1}{N} \sum_{i=1}^N \delta_{(\theta_i, z_i)}(\theta, z).$$

The experiment therefore provides a benchmark for sequential design in a high-dimensional, hierarchical, and partially implicit setting. The design problem is challenging because only a small number of localized measurements are allowed, while the latent image is high-dimensional. Informative policies must therefore learn where to observe the image so as to reveal discriminative regions for the downstream classifier. This directly tests whether an acquisition strategy can align its designs with the final prediction task, rather than merely collecting information about the full latent image.

G.3.1 Baselines, architectures, and downstream objective.

Protocol. We follow the same experimental protocol as in the previous experiments, with the main difference that the label-level observation model is implicit. As a result, EIG bounds are not reported for this task. Acquisition policies are trained using only images from the training set, while final performance is evaluated and reported on both the training and test sets. All downstream classifiers share the architecture reported in Table 27 and are trained with the cross-entropy loss

$$\mathcal{L}_{\text{CE}}(w) = -\mathbb{E}_{(z, \theta), h_T} [\log \pi_a^\psi(z | h_T)],$$

where z denotes the class label and h_T the final acquisition history. For each method, training is monitored on a validation split and stopped by early stopping once validation performance no longer improves. The resulting training budgets and selected hyperparameters are summarised in Tables 28 and 29. To avoid redundancy, these tables report only the hyperparameters that differ from those in Table 13; all omitted entries are kept unchanged. We report the final classification loss and accuracy on both the training and test sets.

Table 27: MNIST masked classification downstream architecture.

Module	Layer	Description	Dimension / Activation
Pair encoder	Input	(ξ_t, y_t)	$\dim(\xi) + \dim(y) / -$
	H1	Fully connected	512 / ReLU
	H2	Fully connected	256 / ReLU
	H3	Fully connected	128 / ReLU
	Output	Fully connected	32 / -
History aggregation	Input	$\{R(\xi_s, y_s)\}_{s=1}^T$	$T \times 32 / -$
	Pooling	Sum or mean pooling	32 / -
Prediction head	Input	Aggregated history	32 / -
	H1	Fully connected	128 / ReLU
	H2	Fully connected	64 / ReLU
	Output	Class logits	$K / -$

Notes. The downstream classifier uses the same encoder–aggregation structure as the acquisition network, with a larger pair encoder to handle the higher-dimensional masked-image observations. The output dimension $K = 10$ is the number of classes.

Architectures. The acquisition policies used by DAD, iDAD, and ACTION-BED rely on the same architectural backbone as in Appendix E. Apart from the task-specific dimensions of the design–observation pairs, the encoding dimensions are kept unchanged with respect to Table 11. The only

Table 28: MNIST masked classification hyperparameters: DAD and iDAD.

Field	DAD	iDAD
Batch size	512	512
Learning rate	5×10^{-4}	10^{-3}
Policy steps	50K	10K
Contrastive samples	$L = 500$	Exact over labels
Downstream steps	50K	–
Downstream LR	10^{-4}	–

Notes. For iDAD, no additional downstream classifier is trained: predictions are obtained directly from the critic logits. The contrastive denominator is computed exactly over the discrete label space. The RANDOM downstream network uses the same hyperparameters as DAD.

Table 29: MNIST masked classification hyperparameters: ACTION-BED and ALINE.

Field	ALINE	Action-BED
Batch size	200	512
Number of queries	128	–
Number of queries, burn-in	128	–
Learning rate	10^{-3}	5×10^{-4}
Total steps	90K	10K
Burn-in steps	10K	–
Warm-up steps	–	0
Joint steps	–	10K

Notes. For ALINE, predictions are obtained directly from the classification head added to the posterior inference network.

additional modification is a sigmoid activation at the output of the emitter network, so that the proposed designs lie in the normalized domain $[0, 1]^2$ before being reprojected onto the pixel grid.

ACTION-BED jointly trains the acquisition policy and downstream classifier for 10K gradient steps using the downstream cross-entropy objective, with no warm-up phase. DAD is trained on sPCE objective, since it requires an explicit likelihood. This objective is defined with respect to the latent image θ rather than the class label z , therefore, its policy seeks designs that discriminate between images, rather than directly between classes. For both DAD and RANDOM, a downstream classifier with the same architecture as in ACTION-BED is then trained separately for 50K gradient steps on trajectories generated by the corresponding policy.

We also compare against iDAD, which adapts the contrastive InfoNCE objective to the implicit class-conditional setting. It jointly learns a design policy π_d^{iDAD} and a critic $U_\psi(z, h_T)$ measuring the compatibility between a candidate label and the observed history. Since the latent parameter space is discrete, the nested expectation appearing in the denominator of the InfoNCE objective can be accurately approximated by summing over all possible labels rather than relying on Monte Carlo negative samples. In practice, this amounts to replacing the usual contrastive denominator by a categorical normalization over the label space,

$$\mathcal{L}_{\text{iDAD}}(\phi, \psi) = -\mathbb{E}_{z, h_T} \left[\log \frac{\exp\{U_\psi(z, h_T)\}}{\sum_{z'=1}^9 \exp\{U_\psi(z', h_T)\}} \right].$$

For ALINE, we keep the native encoder–decoder architecture and modify only the target prediction head. Whereas the original implementation parameterises the posterior by a Gaussian mixture, the masked classification task requires a discrete posterior over labels. We therefore replace the Gaussian-mixture head by a classification head producing categorical logits. The target decoder outputs embeddings z_{target} , with one target token per class. Each token embedding is passed through a small MLP to produce one scalar logit,

$$\ell_k = f_{\text{cls}}(z_{\text{target}, k}), \quad k = 1, \dots, 10,$$

and hence a categorical posterior through a softmax. The posterior inference network is trained with the standard cross-entropy loss. This preserves ALINE’s native token-based architecture while adapting its output layer to the discrete label space.

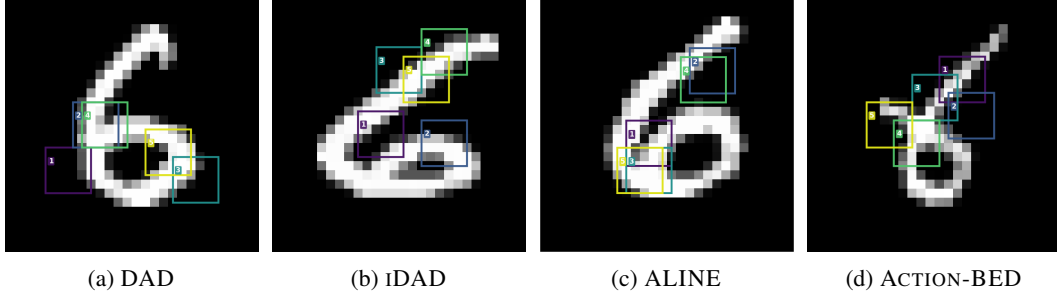


Figure 7: **Comparison of four policy rollouts.** Each panel shows one experimental design trajectory under the trained design policy.

Table 30: **Pair Encoder and Emitter Architectures.**

Module	Layer	Description	Dimension / Activation
Pair encoder	Input	ξ, y	3 / –
	H1	Fully connected	256 / ReLU
	Output	Fully connected	16 / –
Emitter	Input	$R(h_t)$	16 / –
	H1	Fully connected	2 / –
	Output	ξ	2 / –

Similarly, iDAD already includes a critic network which, in this setting, directly scores class labels as a function of the acquisition history. Its outputs can therefore be interpreted as class logits. Consequently, for both iDAD and ALINE, we do not train an additional downstream classifier: predictions are obtained directly from the critic logits for iDAD and from the classification head logits for ALINE.

Evaluation. For MNIST masked classification, loss and accuracy are evaluated after training on the full training and test sets. Standard errors are computed across examples: for the cross-entropy loss, we report the empirical standard error of the per-example losses; for accuracy, we use the Bernoulli standard error associated with the empirical success rate. These uncertainty estimates therefore reflect finite-sample evaluation variability. Additional Figure 7 shows representative design trajectories for the different methods.

Table 31: **Source Location Finding hyperparameters.**

Hyperparameter	DAD	Action-BED
Batch size	2000	2000
Contrastive samples L	2000	–
Policy steps	50K	150K
Downstream steps	150K	–
Warm-up steps	–	50K
Optimizer	Adam	Adam
Adam betas	(0.8, 0.998)	(0.8, 0.998)
Learning rate	5×10^{-4}	7×10^{-4}
LR decay γ	0.95	0.95
LR decay period	2000	2000
Weight decay	0	0
Random seed	42	42

Notes. Contrastive samples are used only for the sPCE-based DAD objective. For ACTION-BED, the acquisition policy and downstream action network are optimised with the downstream loss; warm-up steps are also reported.

G.4 Robustness to Network Architecture

This section reports additional results obtained under the same experimental protocols as in the main experiments, but using lighter neural architectures for both acquisition and downstream inference, and smaller computational budgets. The goal is not to improve upon the main quantitative results, but rather to assess whether the qualitative conclusions are robust to reduced model capacity. In particular, we investigate whether the relative behaviour of ACTION-BED and the information-theoretic baselines DAD and IDAD remains consistent. Across tasks, the experimental setting, simulators, evaluation metrics, and train/test protocols are kept identical to those used in the corresponding main experiments. As expected, this generally leads to weaker absolute performance compared with the main results.

The precise architecture depends on the structure of the task. For exchangeable observation models, we use a DeepSets-style architecture in which each design–observation pair (ξ_s, y_s) is encoded independently and the resulting embeddings are aggregated by sum or mean pooling. For sequential and non-exchangeable simulators, such as the stochastic pendulum, the history is instead processed by a recurrent network, since the likelihood depends explicitly on the previous state. For each task, the downstream action policy π_a^ψ is kept identical to that used in the corresponding main experiment. It is implemented as an MLP taking the final history h_T as input and returning the estimate or action associated with the chosen downstream loss.

G.4.1 Source Location Finding

Training. For the source-location experiment, DAD and ACTION-BED use the same data-acquisition architecture, reported in Table 30. This architecture is the one used in Foster et al. [22] for the corresponding DAD experiment, ensuring that differences in performance are not driven by changes in policy-network capacity. Each design–observation pair (ξ, y) is first processed by a pair encoder, which maps it to a fixed-dimensional embedding. The embeddings associated with the history h_t are then aggregated by a permutation-invariant pooling operation, either sum or mean pooling, and passed to an emitter network that outputs the next design ξ_{t+1} . Both the pair encoder and the emitter are fully connected MLPs.

The RANDOM baseline does not use a learned acquisition network: designs are sampled independently from the initial prior distribution $p(\theta)$. The corresponding training hyperparameters for the learned policies and downstream action networks are reported in Table 31.

Results. The results are reported in Table 32. After training, all policies are evaluated using $L = 100,000$ contrastive samples and a batch size of 2000 rollouts for the sequential PCE and NMC estimates. Downstream losses are computed on independent batches of 2000 rollouts. Uncertainties are reported as ± 1 standard error across rollout histories. We also report the total training time

for each method, including both acquisition-policy and downstream-action-network training when applicable.

Table 32: **Source Location Finding, lightweight architectures.**

Method	Performance (± 1 s.e.)				Compute
	MSE (10^{-2}) $\downarrow$	Log-MSE $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$	Time (min) $\downarrow$
Action-BED (MSE)	2.6 $\pm$ 0.2	-5.59 ± 0.03	9.14 ± 0.04	9.95 ± 0.07	97
Action-BED (Log)	4.2 ± 0.4	-6.39 $\pm$ 0.04	10.36 ± 0.04	12.72 ± 0.03	102
DAD (MSE)	13.6 ± 0.7	-3.72 ± 0.04	10.38 $\pm$ 0.03	12.91 $\pm$ 0.03	283
DAD (Log)	15.5 ± 1.2	-4.02 ± 0.04	10.38 $\pm$ 0.03	12.91 $\pm$ 0.03	285
Random	22.4 ± 0.8	-3.16 ± 0.04	8.27 ± 0.04	8.49 ± 0.05	31

Notes. MSE is reported in units of 10^{-2} . Uncertainties are reported as ± 1 s.e. over 2048 evaluation rollouts. Compute time includes training both the design and downstream action networks.

G.4.2 Training Stability over Training Seeds

To assess the stability of ACTION-BED, we repeated training across multiple random seeds and evaluated the resulting policies under the same protocol. We found that performance remained consistent across runs, with only moderate variation in downstream loss and EIG estimates. Notably, the standard errors across independently trained policies are of a similar order of magnitude to those obtained from rollout-level uncertainty estimates, suggesting that training-seed variability is not substantially larger than evaluation noise. This suggests that the joint optimisation of the design and action policies is not overly sensitive to random initialisation or stochastic training trajectories. In particular, the observed gains over EIG-based baselines in terms of downstream losses are not driven by a single favourable run, but persist across independently trained policies.

Table 33: **Training stability over random seeds on Source Location Finding.**

Method	MSE (10^{-2}) $\downarrow$	Log-MSE $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$
ACTION-BED (MSE)	2.7 ± 0.1	-5.54 ± 0.02	9.04 ± 0.04	9.79 ± 0.06
ACTION-BED (Log)	3.9 ± 0.3	-6.27 ± 0.02	10.21 ± 0.07	12.67 ± 0.09
DAD (MSE)	14.1 ± 0.3	-3.78 ± 0.05	10.36 ± 0.07	12.79 ± 0.05
DAD (Log)	15.7 ± 0.9	-3.96 ± 0.06	10.36 ± 0.07	12.79 ± 0.05
Random	21.2 ± 0.4	-3.19 ± 0.08	8.21 ± 0.02	8.41 ± 0.02

Notes. Uncertainties are reported as one standard error over independently trained policies with 15 different random seeds. For each trained policy, metrics are estimated using the same evaluation protocol.

Table 34: Acquisition architecture for the pendulum tasks.

Module	Layer	Description	Dimension / Activation
Pair encoder	Input	(ξ_t, x_t)	$\dim(\xi) + \dim(x) / -$
	H1	Fully connected	256 / ReLU
	H2	Fully connected	256 / ReLU
	Output	Fully connected	64 / -
History aggregator	Input	$\{R(\xi_s, x_s)\}_{s=0}^t$	$(t+1) \times 64 / -$
	H1	LSTM	64 / -
	H2	LSTM	64 / -
Emitter	Input	Final recurrent state	64 / -
	H1	Fully connected	256 / ReLU
	H2	Fully connected	256 / ReLU
	Output	Design ξ_{t+1}	$\dim(\xi) / \tanh$

Notes. The design dimension is $\dim(\xi) = 1$ for the stochastic single pendulum and $\dim(\xi) = 2$ for the double pendulum. The tanh output is subsequently rescaled to the task-specific control domain.

G.4.3 Dynamical Systems

Architecture. For both pendulum tasks, the acquisition network must account for the temporal dependence of the simulator. We therefore use the same sequential architecture for the stochastic single and double pendulum, inspired by Ivanova et al. [39]. Each design–state pair (ξ_t, x_t) is first embedded by an MLP encoder, and the resulting sequence of embeddings is processed by a two-layer LSTM. The final recurrent state is then passed through an emitter MLP, whose output dimension matches the design dimension of the task. The full acquisition architecture is summarised in Table 34.

The downstream action network is kept identical to that used in the corresponding main experiments. It is implemented as an MLP taking the full trajectory h_T as input and returning the parameter estimate associated with the chosen downstream loss. Unless otherwise stated, all optimisation hyperparameters are also inherited from the main experiments. The only change is the training budget: for the stochastic single pendulum, all policy, downstream, and joint networks are trained for 30K gradient steps; for the double pendulum, all corresponding networks are trained for 50K gradient steps. No downstream-only warm-up phase is used in either task.

The results are reported in Tables 35 and 36. They are obtained using the same evaluation protocol as in the main experiments, with downstream losses computed on independent rollout batches and information-theoretic metrics estimated using sequential PCE and NMC bounds.

Table 35: Stochastic Single Pendulum.

Method	Performance (± 1 s.e.)				Compute
	MSE (10^{-2}) $\downarrow$	Log-MSE $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$	Time (min) $\downarrow$
Action-BED (MSE)	1.0 $\pm$ 0.09	-5.30 ± 0.03	3.35 $\pm$ 0.02	3.40 $\pm$ 0.02	45
Action-BED (Log)	1.1 $\pm$ 0.11	-5.85 $\pm$ 0.04	2.55 $\pm$ 0.01	2.66 $\pm$ 0.01	47
DAD	1.7 $\pm$ 0.16	-5.34 ± 0.05	2.63 $\pm$ 0.12	2.65 $\pm$ 0.11	242
Random	2.90 $\pm$ 0.13	-4.15 ± 0.08	1.39 $\pm$ 0.07	1.42 $\pm$ 0.08	5

Notes. For DAD, downstream performance is reported only for the downstream network trained with the corresponding loss. Uncertainties are reported as ± 1 s.e over 2048 rollout histories. Compute time includes training both the design and downstream action networks when applicable.

G.4.4 MNIST Masked Classification

For the MNIST masked-classification task, we use the same class of acquisition architectures as in the source-location experiment. In particular, the design policies are implemented with a DeepSets-style backbone: each design–observation pair is encoded independently, the resulting embeddings are aggregated over the acquisition history, and an emitter network outputs the next design. The precise architecture is reported in Table 37.

Table 36: Double Pendulum.

Method	Performance (± 1 s.e.)				Compute
	MSE (10^{-3}) $\downarrow$	Log-MSE $\downarrow$	sPCE $\uparrow$	sNMC $\uparrow$	Time (min) $\downarrow$
Action-BED (MSE)	0.7 $\pm$ 0.05	-9.25 ± 0.02	10.40 ± 0.02	14.50 ± 0.13	276
Action-BED (Log)	1.4 ± 0.08	-9.95 ± 0.04	9.90 ± 0.03	12.50 ± 0.07	274
DAD (MSE)	1.0 ± 0.04	-9.32 ± 0.03	10.75 ± 0.03	14.10 ± 0.14	582
Random	2.0 ± 0.09	-6.80 ± 0.05	8.26 ± 0.06	8.41 ± 0.08	58

Notes. For DAD, downstream performance is reported only for the downstream network trained with the corresponding loss.

Table 37: MNIST masked classification acquisition-policy architecture.

Module	Layer	Description	Dimension / Activation
Pair encoder	Input	(ξ_t, y_t)	$\dim(\xi) + \dim(y) / -$
	H1	Fully connected	256 / ReLU
	H2	Fully connected	128 / ReLU
	H3	Fully connected	64 / ReLU
	Output	Fully connected	16 / -
History aggregation	Input	$\{R(\xi_s, y_s)\}_{s=1}^t$	$t \times 16 / -$
	Pooling	Sum pooling	16 / -
Emitter	Input	Aggregated history	16 / -
	Output	Design ξ_{t+1}	2 / Sigmoid

Notes. The design-policy backbone is a DeepSets-style net. A sigmoid activation is used to constrain designs to $[0, 1]^2$ before reprojection onto the pixel grid.

The downstream classifier follows the same encoder-aggregation as summarised in Table 27. For iDAD, we use the same backbone for the design policy but the critic encoder is used as a downstream classifier as in the main experiment.

Training hyperparameters for all methods are kept identical to those used in the corresponding main experiment. The corresponding classification results, including cross-entropy loss and accuracy on both training and test sets, are presented in Table 4.

Table 38: MNIST masked classification.

Method	Performance (± 1 s.e.)				Compute
	Train CE $\downarrow$	Train Acc. (%) $\uparrow$	Test CE $\downarrow$	Test Acc. (%) $\uparrow$	Time (min) $\downarrow$
Action-BED	0.050 $\pm$ 0.002	98.24 $\pm$ 0.04	0.128 $\pm$ 0.006	96.55 $\pm$ 0.13	2.5
DAD	0.398 ± 0.004	83.10 ± 0.03	0.609 ± 0.010	77.75 ± 0.12	18
iDAD	0.113 ± 0.003	96.28 ± 0.07	0.175 ± 0.007	94.76 ± 0.19	5
Random	0.786 ± 0.015	71.78 ± 0.87	0.797 ± 0.015	71.40 ± 0.86	5

Notes. CE denotes cross-entropy loss and Acc. denotes classification accuracy. Metrics are computed after training on the full training and test datasets. Uncertainties are reported as one standard error across examples. Compute time includes training the acquisition policy and, when applicable, the downstream classifier.